PQ-Vault: Provably Quantum-Safe Asset Custody for Unmodified Ethereum

Hassan Khodaiemehr, *Member, IEEE,* Khadijeh Bagheri, and Chen Feng, *Member, IEEE*

Abstract—A quantum computer could recover the key of any Ethereum account whose public key appears on chain. We prove that no sidechain or bridge that leaves Ethereum's rules unchanged can protect such accounts: their funds must move into a contract whose release condition relies only on quantum-safe primitives. We design *PQ-Vault*, a custody system for unmodified Ethereum combining this vault, a quantum-safe proof-of-stake sidechain whose checkpoints carry compact certificates, and an optimistic dispute protocol. We give the first security proof of compact certificates against quantum attackers and a matching attack: with classically chosen parameters, the guarantee falls from 128 to about 60 bits, and a simple rule restores it. We show that a common seed-update rule in hash-chain proof of stake lets an attacker steer chosen slots and profit from splitting its stake, and give an election whose steering gain and inclusion delay are provably bounded against quantum attackers. Since compressing certificates with a pairing-based SNARK breaks quantum security, we use fraud proofs that need a single signature check. For 65,536 validators, a certificate is 0.8 MB, 260 times smaller than all signatures, and verifies in about 20 ms; the vault accepts a checkpoint for about 420,000 gas.

Index Terms—Post-quantum cryptography, blockchain, Ethereum, sidechain, bridge, proof of stake, compact certificates, quantum random oracle model, ML-DSA, Fiat–Shamir.

1 Introduction

Ethereum authenticates every externally owned account (EOA) with ECDSA over secp256k1, and its consensus layer relies on BLS signatures over pairing-friendly curves. Both are broken in polynomial time by Shor's algorithm [1]. Once a cryptographically relevant quantum computer (CRQC) exists, which we call *Q-day*, any account whose public key has appeared on chain can be emptied by an adversary that recovers the secret key and signs a valid transaction [2], [3]. Users and applications therefore need a way to hold value *today* that stays safe after Q-day, using post-quantum (PQ) cryptography, without waiting for changes to Layer 1 (L1).

Four lines of work address this problem, and each contributes an essential piece. *Protocol migration*, through an emergency hard fork that freezes ECDSA and recovers accounts [4], PQ precompiles [5] and hash-based multi-signatures for consensus [6], is the definitive long-term solution, but it depends on community coordination and an uncertain timeline. *PQ ledgers*, such as QRL [7], Abelian [8] and permissioned designs [9], [10], show that PQ consensus and PQ accounts work in production; they secure their own assets rather than those on Ethereum. *PQ smart accounts*, built on Solidity verifiers for ML-DSA and Falcon [11], [12], already run on unmodified Ethereum; each operation pays for an on-chain PQ verification, millions of gas with today's verifiers. *Sidechains, bridges and light-client certificates* [13], [14], [15], [16], [17] make cross-chain verification succinct and amortize it over many users, and PQ sidechain designs for Ethereum [18] combine them with PQ signatures and hash-based proofs [19]. Their security arguments, however, were developed for classical adversaries. To our knowledge, no existing approach yet offers a custody architecture that runs on unmodified Ethereum today, keeps every check on the release path PQ, is proven secure against quantum adversaries end to end, and amortizes its cost across users.

Building such an architecture raises four questions that the classical setting hides. *What can an external layer protect at all?* A PQ proof does not change which transactions L1 accepts, so a quantum adversary spends any asset that an ECDSA signature can spend (Proposition 2). *Are succinct certificates still sound?* Compact certificates [14] are proven in the classical random-oracle model, where grinding with q queries gains a factor q; Grover's algorithm [20] gains a factor of order q^2, and classically chosen parameters keep only about 60 of 128 bits (Corollary 7). *Is PQ leader election fair?* In hash-chain proof of stake (PoS) [7], the next seed depends on who proposes, so a leader can steer it by skipping, and splitting stake pays off (Proposition 9 and Corollary 10); a fix must also hold against superposition queries. *Can verification on L1 be cheap without losing PQ security?* A pairing-based SNARK saves gas but puts a quantum-breakable assumption on the path that L1 executes (Proposition 17).

We present *PQ-Vault*, a custody architecture for unmodified Ethereum that answers these questions, each with a proof and an evaluation (Table 1, Fig. 1): a vault released only by ML-DSA and hash checks, which escrow under a PQ predicate makes necessary (C1); a quantum random-oracle model (QROM) analysis and reveal rule for compact certificates (C2); a steering-resistant, Sybil-proof epoch seed for stake-weighted election, which leaves only an explicitly bounded choice among seeds (C3); and single-reveal fraud

- *H. Khodaiemehr, K. Bagheri, and C. Feng are with the School of Engineering, University of British Columbia (Okanagan Campus), Kelowna, BC, Canada.*
  E-mail: {hassan.khodaiemehr, khadijeh.bagheri, chen.feng}@ubc.ca
- *This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.*

TABLE 1
Contributions of this paper.

Contribution	Key results
C1 Custody (Sections 4 and 5)	No mechanism that leaves L1 unchanged protects ECDSA-controlled assets (Proposition 2); escrow under a PQ predicate is necessary (Corollary 3); vault with a PQ-vetoable identity binding (Proposition 4).
C2 Certificates (Section 6)	QROM bound $12t^2(W_\mathsf{p}/W_\mathsf{s})^{k_\mathsf{rev}} + O(t^3/2^h) + 2\mathsf{Adv}^{\mathsf{uf}}_K$ with explicit constants (Theorem 5); Grover attack: the q^2 factor is necessary (Theorem 6); classical parameters: $\approx$60 proven bits, attack at 2^{-81} (Corollary 7); reveal rule with $\lambda + 2\log_2 q + 4$ and a security budget.
C3 Leader election (Section 7)	Exponential race (Lemma 8); steering gain $m\alpha/(1+(m-1)\alpha)$ (Proposition 9 and Corollary 10); commit–reveal epoch seeds with random reveal order ($\Pr[c \geq x] \leq \mu\alpha^x$) and fallback proposers; share, capture and liveness bounds (Theorem 11, Proposition 13, and Lemma 12) against mildly adaptive quantum adversaries, error below 2^{-178} plus a per-epoch failure below 2^{-40} at $\alpha = 0.3$ (Theorem 15 and Corollary 16).
C4 L1 acceptance (Section 8)	Groth16 wrapper is not PQ (Proposition 17); dispute state machine with single-reveal fraud proofs (Theorem 19); end-to-end custody by induction over checkpoints (Theorem 20).
C5 Evaluation (Section 9)	Dilithium (r3) certificates for 2^8–2^{16} validators (0.8 MB, 260$\times$ smaller), FIPS 204 cross-check; Falcon and hash-based [6] baselines; TCP deployment; simulations of Section 7; Solidity vault (4.2×10^5 gas per checkpoint).

proofs in place of SNARK compression (C4). PQ-Vault needs no change to L1; one checkpoint per epoch, accepted for 4.2×10^5 gas, serves all users; and the evaluation (C5), with real certificates for up to $65{,}536$ validators and a Solidity vault, is released as an open artifact [21]. The appendices contain the full proofs, the complete dispute protocol, and further details of the design and evaluation.

2 Related Work

Quantum threats and PQ ledgers. Shor's algorithm breaks ECDSA, EdDSA and BLS, and Grover's algorithm speeds up preimage search [2]; we adopt the distinction between slow and fast quantum attackers of [3] (Assumption A3). Our binding (Section 5.2) is compatible with the proposed quantum-emergency hard fork [4] but does not depend on it, and Section 4 compares PQ-Vault with PQ smart accounts [11], [12]. QRL [7], Abelian and its PQZK bridge [8], [22], PQFabric [9] and SodsBC [10] secure their own ledgers; to our knowledge, which Ethereum assets an external PQ layer can protect has not been formalized before.

Sidechains, light clients and aggregation. PoS sidechains were formalized in [13]; light-client proofs include compact certificates and Algorand state proofs [14], [15], Mithril [16], and SNARK-based bridges such as zkBridge [17], which rely on pairings [23] or on transparent hash-based proofs [19], [24]. Lattice-based synchronized multi-signatures [25] and hash-based multi-signatures for Ethereum [6] need synchronized or few-time keys, whereas compact certificates work with an unmodified standard scheme and arbitrary weights (Section 9.2). To our knowledge, Theorems 5 and 6 give the first quantum analysis of compact certificates. Like optimistic rollups [26], our dispute game is optimistic; the conjunctive structure of certificate verification makes its fraud proofs one-step.

Leader election and the QROM. Leader election based on verifiable random functions (VRFs) [27], [28] is not PQ unless the VRF is, and lattice VRFs remain costly [29]; hash chains [7] and commit–reveal beacons are PQ alternatives. Grinding and long-range attacks on PoS are well known [30], [31]. In the QROM [32], the soundness error of Fiat–Shamir and of BCS-compiled interactive oracle proofs scales with $t^2\varepsilon$ [33], [34]; we instantiate this lifting for compact certificates and show that the factor q^2 is necessary.

TABLE 2
Positioning against related systems. ✓: provided; ○: partial or assumed; –: not provided.

	Unmod. Ethereum	PQ L1 accept path	QROM proof	Grinding analysis	Sublinear cert	Impl. + gas model
QRL / Abelian [7], [8]	–	✓	–	○	–	–
PQZK bridge [22]	○	–	–	–	–	–
Compact certs [14]	–	–	–	○	✓	○
zkBridge [17]	✓	–	–	n/a	✓	✓
Hash-sig multisig [6]	–	✓	○	n/a	○	○
Our prior work [18]	✓	–	–	–	✓	○
PQ-Vault (this work)	✓	✓	✓	✓	✓	✓

Table 2 positions PQ-Vault against representative systems.

3 Preliminaries

Notation. λ is the security parameter. H is a hash function with h-bit output (Keccak-256 or SHA3-256, with $h = 256$), modeled as a random oracle ρ to which the adversary may make *quantum* (superposition) queries [32]. We write q for the adversary's queries and $t \geq q$ for all queries in an experiment, including those of honest parties and verifiers. ML-DSA hashes internally with SHAKE256, whose padding differs from that of Keccak-256 and SHA3-256; we model it as an oracle independent of ρ. cSHAKE256 with a dedicated customization string, used for the commitments and seeds of Section 7, is a third independent oracle. For weights (w_i), $w(S) = \sum_{i \in S} w_i$. Stakes and weights are integers.

Signatures. We use ML-DSA (FIPS 204 [35], formerly CRYSTALS-Dilithium [36]), designed to be strongly unforgeable [35, Sec. 3.1] under MLWE, MSIS and SelfTargetMSIS [36, Sec. 4]. Our proofs need only existential unforgeability (EUF-CMA), so they hold for hedged and for deterministic signing alike [35, Sec. 3.4]. In the multi-user game with K honest keys, all generated by the challenger, and adaptive corruptions, $\mathsf{Adv}^{\mathsf{uf}}_K$ denotes the advantage of forging for an uncorrupted key; K counts every honest key generated in the experiment, and guessing the target key loses at most a factor K. Public-key/signature sizes are $1312/2420$, $1952/3309$ and $2592/4627$ bytes for ML-DSA-44/65/87.

Merkle trees. Trees are domain-separated (prefix $0x00$ for leaves, $0x01$ for internal nodes), padded to 2^d leaves, and opened with *compact multiproofs*, which contain the siblings of the minimal subtree spanning the opened set P that are not in that subtree: at most $|P|(d - \lfloor \log_2 |P| \rfloor + 1)$ hashes, and far fewer for clustered positions, so we report measured sizes.

QROM tools. We use the database framework of Chiesa, Manohar and Spooner [34]. The oracle is simulated as a compressed oracle [37], and measuring it after the algorithm halts yields a *database* D of input–output pairs. A *database property* is a set of databases. Its *instability* $I(P,t)$ is the largest probability, over databases D with fewer than t entries, inputs x not in D, and a uniform output y, that

Algorithm 1 Certificate verification $\mathsf{Vf}(R_{\mathsf{att}}, M, W_{\mathsf{p}}, \mathsf{cert})$. The verifier state holds W, the number N of positions of the attestor list (zero-weight positions included), and $d = \lceil \log_2 N \rceil$; δ, κ_Q and the design query bound $q = 2^{64}$ are fixed in Section 6. All quantities are integers, and leaves have fixed-length encodings. Comments mark checks that were missing from, or different in, our prior prototype [38].

parse $\mathsf{cert} = (R_{\mathsf{sig}}, W_{\mathsf{s}}, \{(i, pk_i, w_i, \sigma_i, L_i)\}_{i \in P}, \pi_{\mathsf{sig}}, \pi_{\mathsf{att}})$
require $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W$ ▷ W, N from state
$k_{\mathsf{rev}} \leftarrow \lceil (\lambda + 2\log_2 q + \kappa_Q)/\log_2(W_{\mathsf{s}}/W_{\mathsf{p}}) \rceil$ ▷ QROM rule
require $P \subseteq [N]$, $1 \le |P| \le k_{\mathsf{rev}}$ ▷ query bound
for $i \in P$ **do require** $w_i > 0 \wedge L_i + w_i \le W_{\mathsf{s}}$ ▷ interval
end for
for $j = 0, \ldots, k_{\mathsf{rev}} - 1$ **do**
  $c_j \leftarrow$ Eq. (1) with a 128-bit reduction
  require $\exists i \in P : L_i \le c_j < L_i + w_i$ ▷ coverage
end for
require each $i \in P$ covers some c_j ▷ no unused reveal
for $i \in P$ **do require** $\mathsf{ML\text{-}DSA.Vf}(pk_i, M, \sigma_i) = 1$
end for
require π_{sig} opens $\{(\sigma_i, L_i)\}_{i \in P}$ at P under R_{sig}
require π_{att} opens $\{(pk_i, w_i)\}_{i \in P}$ at P under R_{att}
return 1

adding (x, y) to D changes whether the database lies in P. The conditional instability $I(P \,|\, \bar{Q}, t)$ counts only transitions in which neither database lies in a second property Q [34, Def. 7].

Lemma 1 ([34], Lemmas 1 and 9)**.** *Let $\mathcal{A}$ make t quantum queries to a random oracle with n-bit outputs, and let D be the database measured after it halts. (i) For database properties P, Q that do not contain the empty database, $\Pr[D \in P] \le 6t^2\big(I(P \,|\, \bar{Q}, t) + I(Q, t)\big)$. (ii) If the final check of a game reads k oracle values, its winning probability ω_O against the real oracle and the probability ω_D that the check passes with those values read from D satisfy $\sqrt{\omega_O} \le \sqrt{\omega_D} + \sqrt{k/2^n}$.*

When inputs of different kinds have different output lengths, instability is computed with the output length of the queried input, and in (ii) n is the smallest output length among the values read. For the property Col that D contains two inputs with equal h-bit outputs, $I(\mathsf{Col}, t) < t/2^h$ [34, Sec. 2.5]. If $D \notin \mathsf{Col}$, each root in D determines at most one tree of entries of D below it; a leaf that does not occur in D is treated as $\perp$.

Compact certificates [14]. Fix an attestor list $((pk_i, w_i))_{i<N}$ committed in a Merkle tree with root R_{att}, a message M, and a *proven weight* W_{p}. Let S be the set of attestors that signed M, and let $W_{\mathsf{s}} = w(S) > W_{\mathsf{p}}$ be the *signed weight*. The prover assigns to each $i \in S$ the interval $[L_i, L_i + w_i)$, with the L_i accumulated over signers in index order. It commits to the leaves (σ_i, L_i) in a tree with root R_{sig} and derives k_{rev} coins

$$c_j = \rho(\mathrm{DOM} \,\|\, j \,\|\, R_{\mathsf{sig}} \,\|\, R_{\mathsf{att}} \,\|\, W_{\mathsf{s}} \,\|\, W_{\mathsf{p}} \,\|\, N \,\|\, M) \bmod W_{\mathsf{s}}, \tag{1}$$

for $j \in [k_{\mathsf{rev}}]$, using the first 128 output bits. For each coin it reveals the leaf whose interval covers the coin, together with that attestor's leaf in R_{att}. The verifier checks every coin, interval, signature, and Merkle opening (Algorithm 1). In the classical ROM, a prover that knows signatures of total weight at most W_{p} succeeds with probability at most $q(W_{\mathsf{p}}/W_{\mathsf{s}})^{k_{\mathsf{rev}}}$, and [14] accordingly sets $k_{\mathsf{rev}} = \lceil (\lambda + \log_2 q)/\log_2(W_{\mathsf{s}}/W_{\mathsf{p}}) \rceil$.

4 System and Threat Model

4.1 Entities

On unchanged *Ethereum (L1)*, *users* hold an address $Addr_M$ with ECDSA keys (sk_M, pk_M) and ML-DSA keys (sk_S, pk_S); ML-DSA-keyed *validators* stake on the sidechain $\mathcal{S}$; *relayers* post checkpoints to L1; *watchers* (anyone) challenge invalid ones; the *vault* Vault escrows assets (Section 5).

4.2 Adversary

The QPT adversary $\mathcal{A}$ (i) recovers the secret key of any ECDSA or BLS public key it sees after Q-day τ_Q; (ii) makes q quantum queries to ρ ($q \le 2^{64}$ in concrete estimates); (iii) corrupts validators holding at most a fraction $f < 1/3$ of every epoch's stake, *mildly adaptively*: a corruption requested in epoch x takes effect from epoch $x+3$, the validator staying honest and keeping its secrets until then. It may also suppress (DoS) at most B honest proposers per slot, chosen after seeing the schedule. Offline honest validators count as corrupt in the leader-election analysis, whose bounds need corrupt plus offline stake below one third. Past-epoch validators may be corrupted after unbonding, which releases stake only T_{fin} after Vault finalized a checkpoint whose next attestor set excludes the validator (Section 8), so no retired key is in an attestor set the vault still accepts, even if checkpointing stalls; (iv) schedules sidechain messages within the synchrony bound $\Delta_{\mathcal{S}}$; and (v) submits arbitrary L1 transactions and orders the mempool, subject to Assumption A1.

4.3 Assumptions

Assumption A1 (L1 integrity)**.** L1 executes contract code as specified, and any transaction submitted by an honest party is included within Δ_{inc}.

Assumption A2 (PQ primitives)**.** ML-DSA (hedged or deterministic) is EUF-CMA secure against QPT adversaries, and H behaves as a quantum random oracle.

Assumption A3 (Slow key recovery)**.** Recovering an ECDSA secret key from a public key takes longer than Δ_{inc} (no *fast* quantum attacker [3]). It is needed only for transactions that *expose* an ECDSA key (deposits, challenges from fresh EOAs); private transaction channels relax it further.

Assumption A4 (One honest watcher)**.** At least one honest watcher is online whenever a proposal is pending. Within Δ_{chk} of a proposal's inclusion, or of the completion of an open-all posting, it obtains the certificate records or determines that they are unavailable, and evaluates Local on all of them (Section 8). It can pay, per pending proposal, one open-all fee and one challenge bond b_C, and submits timeout and timeoutAll once eligible.

4.4 Security Goals

G1 (custody safety). The vault releases an asset only to a recipient authorized by its legitimate PQ owner, as recorded by the honest sidechain. **G2 (certificate soundness).** No accepted checkpoint certificate for $(R_{\mathsf{att}}, M, W_{\mathsf{p}})$ exists unless attestors of total weight greater than $W_{\mathsf{p}} - w(\text{corrupt})$ honestly signed M. **G3 (fair, steering-resistant leader election).** The adversary's leadership share and its probability of capturing a target slot are bounded explicitly in its stake

fraction and B. **G4 (liveness).** Given a live finality gadget, an honest withdrawal is paid within bounded time, except with the small probability of Corollary 16.

4.5 Why External PQ Layers Cannot Protect EOAs

Proposition 2 (Custody impossibility)**.** *Let* Π *be any protocol that does not modify L1 validity rules. Suppose that at some time* $t \geq \tau_Q$ *an asset* a *is still spendable under the L1 predicate* $\mathsf{ECDSA.Vf}(pk_M, \cdot) = 1$ *with* pk_M *public. Then a QPT adversary can produce an L1-valid transaction spending* a *that L1 cannot distinguish from one signed by the owner. Nothing produced by* Π *can make this transaction invalid.*

Proof sketch. $\mathcal{A}$ obtains sk_M by Shor's algorithm; L1 validity depends only on the predicate and account state, which Π cannot change. The full proof is in Appendix S1. □

Corollary 3. *Consider, on unmodified L1, an asset that must remain* live*, i.e., movable by its owner, and* safe *against* $\mathcal{A}$ *after* τ_Q*. Every predicate that can release it after* τ_Q *must then rest on PQ assumptions and on Assumption A1 alone; in particular, no ECDSA key whose public key is exposed may authorize its release, or change or redirect such a predicate without a PQ-authorized veto. On an account-based L1 this requires holding the asset under a contract predicate* Φ*.*

Proof sketch. Such a key falls under Proposition 2, and only contract code can replace an EOA's ECDSA predicate. The full proof is in Appendix S1. □

PQ-Vault directly instantiates Corollary 3 (scope and consequences: Appendix S1). So does a contract account verifying an ML-DSA or Falcon signature per operation [11], [12], with no committee or watcher but 1.3–8.2×10^6 gas per operation (Table 8), versus 1.0×10^5 gas per exit plus 4.2×10^5 per shared checkpoint for PQ-Vault under the assumptions of Section 8. An EOA delegating to such code (EIP-7702 [39]) stays ECDSA-controlled and falls under Proposition 2.

5 PQ-Vault Design

In Fig. 1, value enters the vault on L1, moves on the PQ sidechain, and leaves only via withdrawals in a *certified checkpoint*, accepted directly or optimistically (Section 8). After τ_Q the vault evaluates only Keccak and, on the dispute path, ML-DSA predicates; the ECDSA-authorized binding registry is off the release path (Section 5.2), so Corollary 3 is met.

5.1 Vault Contract

Vault stores (i) escrowed balances; (ii) a binding registry $\mathsf{Bind}[Addr_M] = \mathsf{H}(pk_S)$; (iii) the latest finalized epoch e^*, its checkpoint message M_{e^*}, the attestor commitment R_{att} with total stake W, $W_{\mathsf{p}} = \lfloor W/3 \rfloor$ and attestor count N, and the finalized withdrawal roots; (iv) pending proposals with their dispute state; and (v) spent withdrawal nullifiers. $\mathsf{deposit}(\beta)$ escrows `msg.value` for the PQ identity $\beta = \mathsf{H}(pk_S)$; propose, with a bond b_R, takes a header $(e, h_e, \rho_e, \omega_e, R_{\mathsf{att}}^{e+1}, \beta_e, W^{e+1}, N^{e+1})$ (block hash, state root, *withdrawals root*, next attestor commitment, binding-operations root, next stake and attestor count) and $(R_{\mathsf{sig}}, W_{\mathsf{s}}, k_{\mathsf{rev}}, C_{\mathsf{cert}})$. It requires $e = e^* + 1$ and k_{rev} equal to Eq. (5), recomputes

$$M_e = \mathsf{H}(\mathrm{CP} \,\|\, chainid \,\|\, \mathsf{Vault} \,\|\, e \,\|\, M_{e^*} \,\|\, h_e \,\|\, \rho_e \,\|\, \omega_e \,\|\, R_{\mathsf{att}}^{e+1} \,\|\, \beta_e \,\|\, W^{e+1} \,\|\, N^{e+1}),$$

on which signatures and coins are checked, snapshots $(R_{\mathsf{att}}, W, W_{\mathsf{p}}, N)$, and opens the proposal for challenge. As the vault computes M_e itself, a finalized ω_e and R_{att}^{e+1} are exactly those signed; C_{cert} commits to the certificate's reveal records (blob data). Further proposal mechanics are in Appendix S2 and Section 8. After the challenge window, with no dispute open, finalize sets $e^* \leftarrow e$, $R_{\mathsf{att}} \leftarrow R_{\mathsf{att}}^{e+1}$, $W \leftarrow W^{e+1}$, $W_{\mathsf{p}} \leftarrow \lfloor W^{e+1}/3 \rfloor$, $N \leftarrow N^{e+1}$, and other proposals for epoch e become stale. withdraw pays *amount* to *recipient* if a Merkle path opens the leaf $\xi = \mathsf{H}(0x00 \,\|\, \mathsf{Vault} \,\|\, chainid \,\|\, recipient \,\|\, amount \,\|\, nonce)$ at its index under a finalized ω_e and ξ is unspent; it marks ξ spent before transferring. Globally unique nonces make ξ a nullifier, paid at most once.

A withdrawal leaf is created only by a sidechain transaction signed with sk_S, so the chain of trust is R_{att}^0 (fixed at deployment) → certified checkpoints → ω_e → a withdrawal leaf → an ML-DSA signature; ECDSA is never consulted. Recipients should be fresh addresses or PQ smart accounts (Proposition 2).

5.2 Time-Anchored, PQ-Vetoable Identity Binding

Replacing the ZK binding of [18] (Appendix S2), we authenticate the binding classically *while classical authentication is still meaningful*, and freeze it afterwards. *Bind:* if no binding exists, $Addr_M$'s call $\mathsf{bind}(\mathsf{H}(pk_S))$ takes effect immediately. *Rebind:* a later call creates the (single) pending rebinding of the address, with a fresh counter n and activation time $\tau_{\mathsf{act}} = \tau_{\mathsf{req}} + D$; applyRebind succeeds only at a timestamp strictly greater than τ_{act} and only if n is not cancelled. *Veto:* the current PQ identity ML-DSA-signs $\mathsf{cancel}(\mathsf{Vault}, Addr_M, n)$ on the sidechain; it reaches L1 in the root β_e of a certified checkpoint, and $\mathsf{applyCancel}(Addr_M, n, e, \pi)$, callable by anyone, checks that π opens it under the finalized β_e and marks n cancelled, effective if included by τ_{act}. *Canary:* a bounty on an address with unknown discrete logarithm, claimable by an ECDSA signature on a fixed digest, permanently disables all ECDSA-authorized registry operations once quantum attacks are demonstrated (Appendix S2).

Proposition 4 (Binding integrity)**.** *Suppose* $\mathsf{Bind}[Addr_M] = \mathsf{H}(pk_S)$ *was set by* $Addr_M$*'s owner, and let* $\Delta_{\mathsf{v}} = \Delta_{\mathsf{obs}} + \Delta_{\mathsf{sc}} + \Delta_{\mathsf{cp}} + \Delta_{\mathsf{apply}}$*, where* Δ_{obs} *bounds the owner's time to observe a pending rebinding and sign,* Δ_{sc} *bounds sidechain inclusion of its transaction,* Δ_{cp} *bounds the time from inclusion to finalization of the carrying checkpoint, including disputes and the* finalize *call (Theorem 19), and* Δ_{apply} *bounds preparing and including* applyCancel*. Suppose* $D > \Delta_{\mathsf{v}}$*, the owner is online at least once in every window of length* $D - \Delta_{\mathsf{v}}$*, these bounds hold, and Assumptions A1 to A4 and the timing conditions of Theorem 19 hold. By Corollary 16 and Theorem 19,* Δ_{sc} *and* Δ_{cp} *fail only with the probabilities given there. Then a QPT adversary can change the binding only by forging an ML-DSA signature, by breaking certificate soundness, or before the owner's first binding transaction is included.*

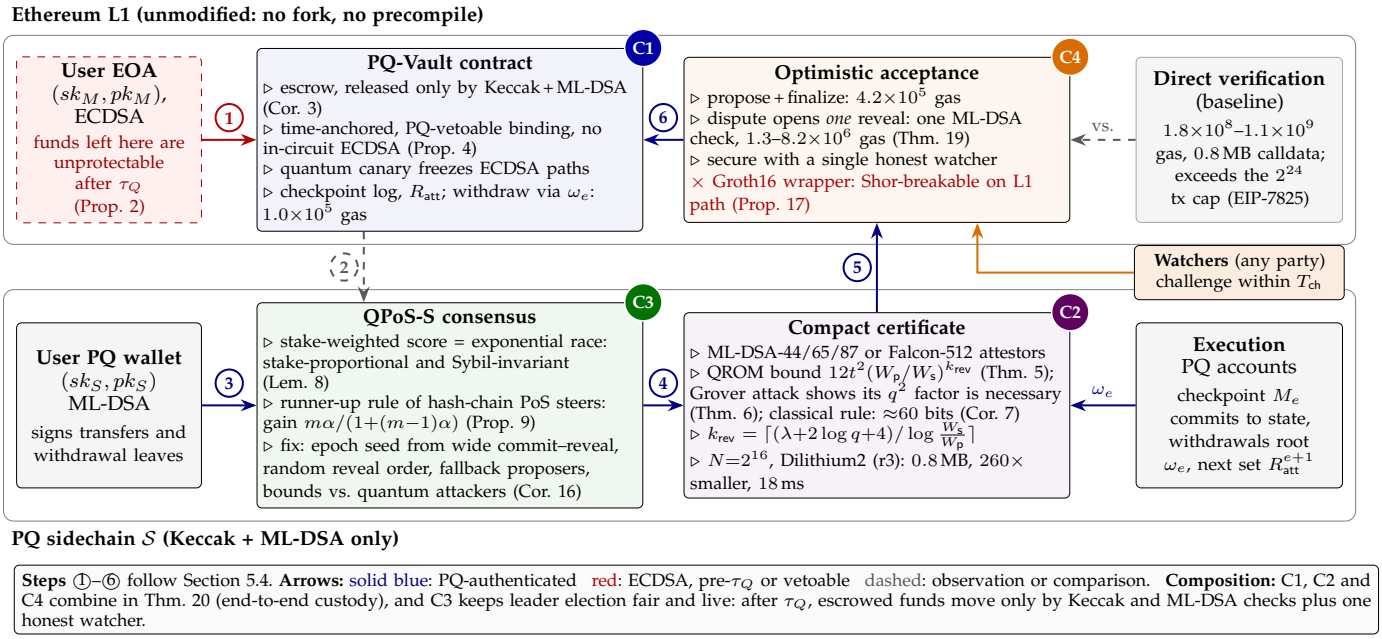


Fig. 1. PQ-Vault architecture (QPoS-S: quantum-safe proof of stake). Badges C1–C4: contributions of Table 1; gas measured with Foundry (Section 9); grey: baseline avoided by C4.

Proof sketch. Any change after the first binding needs a rebinding pending for D, during which the owner is online with at least Δ_v remaining and gets a cancellation of n applied by τ_{act}, before applyRebind can succeed. Preventing this requires an ML-DSA-signed operation the owner never signed (a forgery), or a finalized checkpoint omitting final binding operations, which no honest validator signs (Theorems 5 and 19). After the canary fires, the binding cannot change at all. The full proof is in Appendix S2. □

No later theorem relies on the registry or the canary (Appendix S2).

5.3 Sidechain Protocol

The sidechain runs QPoS-S (quantum-safe proof of stake), the steering-resistant stake-weighted election of Section 7. Each epoch has k slots; validator i has a unique 64-bit identifier ID_i, and epochs are 64-bit numbers. With its ML-DSA-signed registration for epoch e, which fixes its stake w_i, validator i commits to a uniform $r_i \in \{0,1\}^{512}$ by $a_i = \mathsf{H}_{1536}(\text{COMMIT} \,\|\, ID_i \,\|\, e \,\|\, r_i)$, with H_n the n-bit domain-separated cSHAKE256 (Appendix S2). Registrations for epoch e must be final before the reveal window of epoch $e-2$ opens. Reveals for epoch e are ordinary transactions in the last $\ell = \ell_0 + L_t\ell_s$ slots of epoch $e-1$. The reveal *order* is a stake-weighted random permutation: with $\kappa_i = -\log_2 u(\mathsf{H}(\text{ORDER} \,\|\, e \,\|\, ID_i \,\|\, \mathfrak{S}^{(e-1)}))/w_i$, rank 1 goes to the smallest κ_i. The L_t best-ranked identities form the *tail* and reveal last, in reverse rank order, each in its own sub-window of ℓ_s slots; all others reveal in a common first phase of ℓ_0 slots. A reveal is valid only if it opens a_i and is included in its owner's phase or sub-window, and honest validators broadcast r_i only when that starts. Single-slot finality (as in Tendermint-style BFT gadgets) finalizes each reveal-window block before the next slot, so earlier reveals are final when a sub-window starts. A validator missing its epoch-e reveal is suspended (not slashed) from epoch $e+3$ until it re-registers. QPoS-S uses $L_t = 25$, $\ell_s = \ell_0 = 52$, so $\ell = 1{,}352$, and $k \geq 2{,}000$. The reveal list X, encoding the revealed (ID_i, r_i) as fixed-length fields in increasing identifier order, determines the seed

$$\mathfrak{S}^{(e)} = \mathsf{H}_{512}(\text{SEED} \,\|\, e \,\|\, X). \tag{2}$$

In slot $j \in \{1,\dots,k\}$ of epoch e, validator i's score is

$$s_{i,j} = \frac{-\log_2 u\big(\mathsf{H}(\text{SCORE} \,\|\, e \,\|\, j \,\|\, ID_i \,\|\, \mathfrak{S}^{(e)})\big)}{w_i}, \tag{3}$$

with $u(x) = (\text{int}(x)+1)/2^h$. The lowest score leads (ties: smaller identifier); the epoch's schedule is computable once the seed is final. Ranks $2,\dots,L+1$ are *fallback proposers*: the proposer of slot j is the best-ranked identity among ranks $1,\dots,L+1$ whose ML-DSA-signed block arrives within its deadline $r\Delta_{\mathcal{S}}$ for rank r, and an honest proposer includes every pending transaction, including reveals, received $\Delta_{\mathcal{S}}$ earlier. The fallback changes who proposes, never the seed, so it opens no steering channel. QPoS-S uses $L = 3$. These sizes keep commitments statistically binding and hiding errors small in the QROM (Theorem 15; overhead in Appendix S2). Unlike [7], [18], [38], the seed *does not depend on who proposed earlier blocks*; Section 7 shows why this matters.

Blocks are signed with ML-DSA. Every N_{cp} blocks the leader assembles a checkpoint message M_e. *Honest-signing rule:* an honest validator signs M_e only if (i) the prefix it commits to is final (any standard BFT finality gadget with $f < 1/3$ fits) and extends M_{e-1}; (ii) every state transition is valid under the execution rules, which credit each final deposit event once, debit a withdrawal's amount from its owner's balance, and accept a transfer or withdrawal only with a valid ML-DSA signature of the owner's sk_S on the full operation; (iii) ω_e contains exactly the leaves of the withdrawals executed in epoch e, each with a fresh *nonce*;

and (iv) R_{att}^{e+1} lists the bonded validators of epoch $e+1$ with their stakes, each at one position and with a registered key whose expanded form the validator has checked. Any party can aggregate signatures into a certificate (Algorithm 1). With $W_{\mathsf{p}} = \lfloor W/3 \rfloor$ and $f < 1/3$, a sound certificate implies that at least one honest validator signed M_e, hence that M_e satisfies the rule.

Trust in the attestor set. The vault checks certification, not validity: over a third of the attestor stake could certify an arbitrary M_e, whereas $W_{\mathsf{p}} = 2W/3$ would require more than two thirds at about $4\times$ the reveals and $2.8\times$ the certificate size (Table 4 and Section 9). Custody safety therefore rests on corrupt stake staying below one third of every accepted attestor set (Section 4) (economics: Appendix S2).

5.4 End-to-End Flow

The six steps are: (1) deposit into Vault; (2) after L1 finality, validators credit $\mathsf{H}(pk_S)$; (3) PQ-signed transfers on $\mathcal{S}$; (4) an sk_S-signed withdrawal creates a leaf in ω_e; (5) a relayer proposes (M_e, cert_e), finalized after T_{ch}; (6) anyone calls withdraw with the leaf's Merkle proof. Latency is $\Delta_{\mathsf{cp}} =$ checkpoint interval + certification time + finalization time, the last being $T_{\mathsf{ch}} + \Delta_{\mathsf{inc}}$ without disputes and at most $T_{\mathsf{fin}} + \Delta_{\mathsf{inc}}$ with them (Theorem 19), if finalize is submitted once eligible. As a checkpoint may wait for its predecessor ($e = e^* + 1$), in the worst case $\Delta_{\mathsf{cp}} \le$ checkpoint interval + certification time $+ 2(T_{\mathsf{fin}} + \Delta_{\mathsf{inc}})$, provided the checkpoint interval is at least $T_{\mathsf{fin}} + \Delta_{\mathsf{inc}}$ (used in Proposition 4; certification time: Section 9).

6 Compact Certificates Against Quantum Provers

6.1 Security Game

Definition 1 (CERT-FORGE)**.** The challenger generates ML-DSA key pairs for honest attestors on request, at most K in total. The adversary $\mathcal{A}$ fixes an attestor list $((pk_i, w_i))_{i<N}$ with integer weights, adaptively and possibly after signing queries. Each position carries either the key of one honest attestor (at most one position per honest key) or a key *registered* by $\mathcal{A}$; registered positions are corrupt. The challenger commits to the list with root R_{att}, and $W = \sum_i w_i$. $\mathcal{A}$ may corrupt honest attestors and then receives their secret keys; C is the set of positions that are registered or corrupted when $\mathcal{A}$ halts. At any time, $\mathcal{A}$ may ask an uncorrupted honest attestor i to sign a message of its choice; Q_i is the set of messages i signed. This models honest validators signing the checkpoints they consider final, whenever they do so. $\mathcal{A}$ makes q quantum queries to ρ and outputs (M^*, cert^*). It wins if $\mathsf{Vf}(R_{\mathsf{att}}, M^*, W_{\mathsf{p}}, \mathsf{cert}^*) = 1$ and

$$w\big(C \cup S_{M^*}\big) \le W_{\mathsf{p}}, \qquad S_{M^*} = \{i \notin C : M^* \in Q_i\}.$$

Winning means that the certificate claims more support than exists; the corrupt/honest split exists only in the analysis. If $w(C) \le W_{\mathsf{p}}$, an accepted certificate that does not win implies that at least one honest attestor signed M^*. $W_{\mathsf{p}} = \lfloor W/3 \rfloor$ meets this for every $f < 1/3$.

6.2 QROM Soundness

For a claimed signed weight $W_{\mathsf{s}} > W_{\mathsf{p}}$ and k_{rev} reveals, define the per-commitment soundness error

$$\varepsilon(W_{\mathsf{s}}) = \Big(\frac{W_{\mathsf{p}}}{W_{\mathsf{s}}} + \frac{W_{\mathsf{s}}}{2^{128}}\Big)^{k_{\mathsf{rev}}(W_{\mathsf{s}})}. \tag{4}$$

The additive term accounts for the 128-bit modular reduction in Eq. (1). The verifier accepts only $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W$ (Algorithm 1), with $W < 2^{64}$ from its state; let $\varepsilon^* = \max \varepsilon(W_{\mathsf{s}})$ over this range.

Theorem 5 (QROM soundness of compact certificates)**.** *Let $d = \lceil \log_2 N \rceil$, let $q_V = k_{\mathsf{rev}}(2d+3)$ with $k_{\mathsf{rev}} = k_{\mathsf{rev}}((1+\delta)W_{\mathsf{p}})$, let $t = q + q_V + 2^{d+1}$, and let h be the smallest output width of the oracles and trees involved. For every adversary making q quantum queries to ρ and any number of signing queries,*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{CERT\text{-}FORGE}} \le 12t^2\varepsilon^* + \frac{48t^3 + 2q_V}{2^h} + 2\mathsf{Adv}_K^{\mathsf{uf}}.$$

Proof sketch. Following [24], [34], we merge the k_{rev} coins of Eq. (1) into one wide oracle (one query simulates each query) and apply Lemma 1(i) to a fixed database property P: some coin entry is *light* (valid positions below its roots weigh at most W_{p}) and *hit* (all coins covered), with instability at most $\varepsilon^* + 2t/2^h$. P depends neither on the signing history nor on C. If $\mathcal{A}$ wins, either $D \in P$ or D holds an EUF-CMA forgery, extracted by measuring D after $\mathcal{A}$ halts. With $I(\mathsf{Col}, t) < t/2^h$, this gives $\omega_D \le 6t^2\varepsilon^* + 24t^3/2^h + \mathsf{Adv}_K^{\mathsf{uf}}$ for the database game, and Lemma 1(ii) with the q_V values read by the verifier gives $(\sqrt{\omega_D} + \sqrt{q_V/2^h})^2 \le 2\omega_D + 2q_V/2^h$, which is the bound. The full proof is in Appendix S3. □

The proof needs no entropy of honest signatures, and for $W_{\mathsf{p}} \ge W/4$, $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W < 2^{64}$ and $\delta = 0.1$, the reduction term changes $\varepsilon(W_{\mathsf{s}})$ by a factor of at most $1 + 2^{-51}$ (Appendix S3).

6.3 The Quadratic Dependence on q Is Necessary

Theorem 6 (Quantum grinding attack)**.** *Let $d = \lceil \log_2 N \rceil$ and $c_{\mathsf{ev}} = d + 1 + k_{\mathsf{rev}}$. Suppose the adversary corrupts C with $a = w(C) \le W_{\mathsf{p}}$, receives no honest signatures, and some position outside C or some padding position exists. Fix any W_{s} with $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W$, let $\beta = |\{y < 2^{128} : y \bmod W_{\mathsf{s}} < a\}|/2^{128}$ and $\varepsilon_C = \beta^{k_{\mathsf{rev}}}$; then $\varepsilon_C \ge (a/W_{\mathsf{s}})^{k_{\mathsf{rev}}}$. If $q \ge 2^{d+1} + c_{\mathsf{ev}}$, there is a quantum adversary making at most q queries that wins* CERT-FORGE *with probability at least*

$$\frac{4}{\pi^2}(2T_G+1)^2\varepsilon_C\Big[1 - (2T_G+1)^2\big(\varepsilon_C + (d+2)2^{-h}\big)\Big],$$

where $T_G = \lfloor (q - 2^{d+1} - c_{\mathsf{ev}})/(2c_{\mathsf{ev}}) \rfloor$.

Proof sketch. The adversary packs the intervals of C into $[0, a)$ (only revealed intervals are checked) and salts one other position; a salt is marked if all coins of the resulting root fall below a. Grover search finds one; the marked fraction ν has $\mathbb{E}[\nu] = \varepsilon_C$ and $\mathbb{E}[\nu^2] \le \varepsilon_C^2 + \varepsilon_C(d+2)2^{-h}$, which bounds the loss when ν is large. The full proof is in Appendix S3. □

Corollary 7 (Classical parameters are insufficient)**.** *Consider the classical rule $k_{\mathsf{rev}}^{\mathsf{cl}} = \lceil (\lambda + \log_2 q)/\log_2(W_{\mathsf{s}}/W_{\mathsf{p}}) \rceil$ and $w(C) = W_{\mathsf{p}}$. The grinding term of Theorem 5 then guarantees only about $\lambda - \log_2 q - \log_2 12$ bits, and the attack of Theorem 6 succeeds with probability about $(4/\pi^2)\, q^2 (W_{\mathsf{p}}/W_{\mathsf{s}})^{k_{\mathsf{rev}}}/c_{\mathsf{ev}}^2$. For*

TABLE 3
Lifetime security budget of Theorem 20 in the query model, for $T \le 2^{64}(1+2^{-20})$ total queries (honest and verification queries included) and $E = 2^{20}$ epochs. Values are $-\log_2$ of upper bounds; T is as in Section 6.4.

Term	256-bit (deployed)	384-bit
Grinding $E \cdot 12T^2\varepsilon^*$, Eq. (5) with $\lambda = 128$	108	108
Grinding, rule with $\lambda + \log_2 E = 148$ ($k_{\mathsf{rev}} = 208$ at 2.55)	128	128
Commitments $E(60T^3 + 2q_V + 2q_C)/2^h$	38.1	166.1
Signatures $2E\mathsf{Adv}^{\mathsf{uf}}_{N_{\mathsf{tot}}} + \mathsf{Adv}^{\mathsf{uf}}_{U}$	ML-DSA assumption	
Finality $\Pr[\mathsf{Fin}]$	gadget assumption	

$\lambda = 128$, $q = 2^{64}$, $d = 16$, $W_{\mathsf{s}}/W_{\mathsf{p}} = 2.55$ *and* $k_{\mathsf{rev}} = 143$, *the guarantee is* 61.5 *bits (*60.4 *before rounding* k_{rev} *up) and the attack succeeds with probability* $2^{-81.1}$. *Both are far from the intended* 2^{-128}.

The upper bound and the attack differ by a factor of about $3\pi^2 c_{\mathsf{ev}}^2$ ($2^{19.5}$ at these parameters), which is independent of q, so the quadratic dependence on q is unavoidable. The attack needs corrupt weight close to W_{p}: at $W_{\mathsf{s}} = 0.85W$ with $w(C) = 0.30W$ it reaches only $2^{-102.8}$. Depth limits [40] reduce Grover's gain in practice, so our rule is conservative. Algorand's target $k + 2q$ [15] matches our rule up to κ_Q and, modelling its hashes as random oracles, certifies about 124 bits; the classical $k + q$ rule of [14], used by the prototype of [38], falls under Corollary 7 (Appendix S3).

6.4 Parameter Rule and Security Budget

Write $\kappa_Q = \lceil \log_2 12 \rceil = 4$. Requiring $12t^2\varepsilon^* \le 2^{-\lambda}$ in Theorem 5, with $t \le q(1+2^{-20})$ for $q = 2^{64}$, gives the reveal rule

$$k^{\mathsf{Q}}_{\mathsf{rev}} = \left\lceil \frac{\lambda + 2\log_2 q + \kappa_Q}{\log_2(W_{\mathsf{s}}/W_{\mathsf{p}})} \right\rceil, \tag{5}$$

since $12(1+2^{-20})^2(1+2^{-51}) < 16$ for $W_{\mathsf{p}} \ge W/4$. It controls only the *grinding* term; the exact rule for a total query budget T (all L1-path hash queries) is in Appendix S3.

The other terms need separate budgets; the commitment term, with h the *smallest* output width, is an upper bound like the generic collision bound [41], not an attack (Appendix S3). Table 3 combines all terms of Theorem 20 for the deployed Keccak-256 vault and a widened 384-bit variant (not implemented). Keccak-384 certificate trees enlarge Dilithium2 (r3) certificates by 4–7% at unchanged verification time ($N = 2^{12}$–2^{16}, Table 6).

The deployed bound is dominated by the 256-bit commitment term, shared by every Keccak-256 Merkle structure on Ethereum. A lifetime target of 2^{-128} for every term requires both the widened trees and the rule with $\lambda + \log_2 E$; our prototype targets $\lambda = 128$ for the grinding term per checkpoint and does not claim a 128-bit lifetime custody guarantee.

With $\lambda = 128$ and $q = 2^{64}$, the rule increases k_{rev} by a factor of $260/192 = 1.35$ over the classical rule; because certificate size grows with the number of *distinct* reveals, the measured size overhead is smaller (Section 9). Table 4 lists k_{rev} for typical ratios. With $W_{\mathsf{p}} = \lfloor W/3 \rfloor$ and 85% participation, $W_{\mathsf{s}}/W_{\mathsf{p}} = 2.55$, whereas certifying a two-thirds threshold ($W_{\mathsf{s}}/W_{\mathsf{p}} = 1.28$) multiplies the cost by almost 4. This is why we certify finality, which needs only $W_{\mathsf{p}} = \lfloor W/3 \rfloor$ under $f < 1/3$, rather than a supermajority vote.

TABLE 4
Reveal count k_{rev} for $\lambda = 128$ and $\log_2 q = 64$.

$W_{\mathsf{s}}/W_{\mathsf{p}}$	1.25	1.5	2	2.55	3	4
classical [14]	597	329	192	143	122	96
QROM, Eq. (5) (κ_Q=4)	808	445	260	193	165	130

Implementation pitfalls. Three omissions in the prototype of [38] each made certificates forgeable: (i) coin coverage was not checked; (ii) interval endpoints were accumulated over all attestors rather than signers and were not checked; and (iii) a heuristic $k_{\mathsf{rev}} \propto (1 - W_{\mathsf{p}}/W_{\mathsf{s}})$ decreased as $W_{\mathsf{s}} \to W_{\mathsf{p}}$, whereas soundness requires $k_{\mathsf{rev}} \propto 1/\log(W_{\mathsf{s}}/W_{\mathsf{p}})$. A fourth check, $|P| \le k_{\mathsf{rev}}$ with every reveal covering a coin, keeps the verifier's query count bounded as Theorem 5 assumes. Algorithm 1 includes all four, and our implementation tests every check negatively.

7 Leader Election: Fairness and Steering

Total stake is 1; the adversary holds stake α (possibly split across identities, including offline honest validators). We analyze an *ideal experiment* with independent uniform scores and prove that the QPoS-S seed of Section 5 realizes it against quantum adversaries. Full proofs are in Appendix S4.

7.1 Stake-Weighted Scores Form an Exponential Race

Lemma 8 (Exponential race). *Let $U_1, \dots, U_n$ be independent and uniform on $(0,1]$, and let $s_i = -\log_2 U_i / w_i$. Then the s_i are independent, $s_i \sim \mathrm{Exp}(w_i \ln 2)$, so (i) $\Pr[i$ has the lowest score$] = w_i/W$, and (ii) splitting stake w into identities w', w'' with $w' + w'' = w$ leaves the coalition's winning probability unchanged (Sybil invariance). With h-bit values $u_i = \lceil U_i 2^h \rceil / 2^h$ and ties broken by the smaller identity, the winner differs from the continuous winner with probability at most $N(N-1)2^{-h}$.*

Proof sketch. $-\ln U \sim \mathrm{Exp}(1)$, and the minimum of exponentials is attained at i with probability proportional to its rate. The full proof is in Appendix S4. □

Lemma 8 applies the weighted-sampling identity of [42] to the score of [7]; it concerns *unselected* uniforms only.

7.2 Steering Through Proposer-Dependent Seeds

In [7], [18], scores derive from private hash-chain values, the seed is updated as $\mathfrak{S}_{j+1} = \mathsf{H}(\mathfrak{S}_j \,\|$ reveal of the *actual* proposer of $j)$, and a skipped leader is replaced by the runner-up. A coalition holding the top L positions can thus choose among $L+1$ seeds, knowing its *own* next-slot scores (Appendix S4).

Proposition 9 (m-way steering). *Suppose that, before slot T, the adversary can choose $\mathfrak{S}_T$ among m candidates; that under each candidate its own best score is a fresh $\mathrm{Exp}(\alpha)$ variable (rates in units of $\ln 2$), independent across candidates, while the honest best score is $\mathrm{Exp}(1-\alpha)$ and unobserved. Picking the candidate that minimizes its own best score is optimal and leads slot T with probability $p_m(\alpha) = m\alpha/(1+(m-1)\alpha)$.*

Proof sketch. As a function of its best score, the adversary's winning probability is $\mathrm{Beta}(\alpha/(1-\alpha),1)$-distributed; the maximum of m copies has mean $p_m(\alpha)$ (Appendix S4). □

Corollary 10 (Steering under the runner-up rule)**.** *Consider proposer-dependent seeds in the race without threshold of [7], and a single-identity adversary that uses only its decision at slot $T-1$. (a) If it observes the honest runner-up's reveal before its publish-or-skip decision is irrevocable (e.g., rushing), it captures slot T with probability $\alpha+\alpha^2(1-\alpha)/(1+\alpha)$. (b) Otherwise, it captures slot T with probability $\alpha\big(\alpha+(1-\alpha)\alpha^{1/(1-\alpha)}\big)+(1-\alpha)\alpha$. Both exceed α. Splitting into more identities raises the probability of holding the top $L\geq 2$ ranks, hence the option count, so Sybil invariance fails.*

Proof sketch. Leading slot $T-1$, the adversary applies Proposition 9 with $m=2$ in case (a), and gains $\mathbb{E}[\max(p,\alpha)]>\alpha$ against a fresh seed in case (b) (Appendix S4). □

For $\alpha=0.3$ the capture probability is 0.348 ($+16\%$) in case (a) and 0.338 in case (b); $p_{16}(0.1)=0.64$. Target slots matter because roles attach to slot indices, for example the N_{cp}-th block that fixes a checkpoint, or the slot whose block includes or censors a dispute-relevant transaction.

7.3 Epoch Seeds: Ideal Model

In QPoS-S (Section 5), each validator reveals its committed secret r_i in the previous epoch's reveal window, in an order drawn from the previous seed; the epoch seed hashes all reveals. Throughout, $\alpha<1/3$.

In each slot, the scores rank all identities as a stake-weighted sample without replacement (Lemma 8, [42]). Ranks $1,\dots,L+1$ may propose; the best-ranked one whose block arrives in time proposes. The adversary may suppress at most $B\leq L$ honest identities per slot. Call a slot *guaranteed* if its first $B+1$ ranks are honest; a guaranteed slot has an honest proposer whatever the adversary suppresses. Let $w_{\max}$ be the largest honest stake, $\alpha_B=\alpha/(1-Bw_{\max})$ and $p_B=1-(1-\alpha_B)^{B+1}$; without denial of service, $p_0=\alpha$.

Let i^* be the best-ranked honest identity of the tail (it reveals last among honest tail identities), τ^* the first time r_{i^*} leaves validator i^*, and c the number of (adversarial) identities ranked above i^*, which reveal after it. The adversary decides which of them reveal, so the reveal set takes at most $m=2^c$ values. Let Fail be the event that the tail contains no honest identity, or that the common phase or some tail sub-window has no guaranteed slot after its first slot. It depends only on the order and the previous schedule, both fixed before τ^*.

Definition 2 (Ideal seed experiment)**.** An experiment is *m-ideal* if, at time τ^*, it samples for each of the at most m possible reveal sets b an independent uniform table of scores and next-epoch order keys, independent of everything before τ^*; it makes all tables available to the adversary; and every party uses the table of the reveal set that actually results.

Theorem 11 (Leader election in the ideal model)**.** *In any m-ideal experiment with continuous scores: (a) Given the reveal set, the proposer ranking of each slot is a fixed public function of the table. No adversarial action within the epoch (skipping, withholding proposals, choosing among its identities) changes any ranking, except by removing the adversary's own identities. If $m=1$, slot leaders are independent across slots and distributed as in Lemma 8. (b) The adversary chooses the table among at most $m\leq 2^c$ candidates by withholding a subset of its late reveals. A slot is not guaranteed with probability at most p_B under each candidate, and the adversary can propose only in such slots. It proposes in a fixed target slot with probability at most $1-(1-p_B)^m$, and its expected fraction of the k slots of the epoch is at most $p_B+\sqrt{c\ln 2/(2k)}$. (c) The probability that none of n specified slots is guaranteed is at most $\min\{1,mp_B^n\}$. For a window of n consecutive slots whose start the adversary may choose after seeing the schedule, it is at most $\min\{1,(k-n+1)mp_B^n\}$.*

Proof sketch. Each of the first $B+1$ ranks is adversarial with probability at most α_B; part (b) uses independence across candidates, part (c) a union bound over candidates and window starts, and the share uses the sub-Gaussian maximal inequality (Appendix S4). □

Fallback proposers matter only under denial of service. Without them ($L=0$), suppressing the leader of every slot leaves the sidechain without honest blocks; with $L\geq B$, only non-guaranteed slots can lose their honest proposer. Part (c) deliberately does not multiply per-slot probabilities under the selected table, which would be invalid after seed selection.

Lemma 12 (Reveals are included)**.** *Suppose that an honest proposer includes every reveal it received $\Delta_{\mathcal{S}}$ before its slot, that messages are delivered within $\Delta_{\mathcal{S}}$ and a slot lasts at least $(L+2)\Delta_{\mathcal{S}}$, and that the finality gadget finalizes each block of the reveal window before the next slot starts. If a phase or sub-window of ℓ slots has a guaranteed slot after its first slot, every honest reveal of that phase or sub-window is included. Under each candidate table of the epoch that contains the window, this fails with probability at most $p_B^{\ell-1}$.*

Proof sketch. A reveal sent when its window starts reaches every later honest proposer, a guaranteed slot has one, and slots are independent under each candidate (Appendix S4). □

Proposition 13 (Late reveals are rare)**.** *Consider consecutive m-ideal epochs, each with respect to its own c, starting from an epoch with $c=0$. Let $\mu=(1-2\alpha)/(1-3\alpha)$, and let OK_e be the event that Fail occurs in none of the epochs up to e. Then, for every epoch e and every $x\geq 1$,*

$$\Pr[c_e\geq x\wedge\mathsf{OK}_e]\leq\mu\alpha^x,\qquad \mathbb{E}[2^{c_e}\mathbf{1}_{\mathsf{OK}_e}]\leq\mu,$$
$$\Pr[\mathsf{OK}_{e-1}\wedge\neg\mathsf{OK}_e]\leq\delta_{\mathsf{LE}},$$

where $\delta_{\mathsf{LE}}=\mu(\alpha^{L_t}+L_t p_B^{\ell_s-1}+p_B^{\ell_0-1})$. The adversary proposes in a fixed target slot of epoch e with probability at most $\pi_{\mathsf{cap}}+e\delta_{\mathsf{LE}}$, where $\pi_{\mathsf{cap}}=p_B+\sum_{x\geq 1}\min\{1,\mu\alpha^x\}\big((1-p_B)^{2^{x-1}}-(1-p_B)^{2^x}\big)$.

Proof sketch. By mild adaptivity, registrations and the corrupt set of epoch e are fixed before the candidate tables of epoch $e-1$ are sampled, and under each candidate the first x ranks of the order are adversarial with probability at most α^x. Summing over the m_{e-1} candidates gives $\mathbb{E}[2^{c_e}\mathbf{1}_{\mathsf{OK}_e}]\leq 1+\mathbb{E}[2^{c_{e-1}}\mathbf{1}_{\mathsf{OK}_{e-1}}]\,\alpha/(1-2\alpha)$, whose fixed point is μ (Appendix S4). □

For $B=0$ and $\alpha=0.1,0.2,0.3$: $\mu=1.14,1.5,4$; $\pi_{\mathsf{cap}}=0.112,0.265,0.622$; and, as $\mathbb{E}[c]\leq\mu\alpha/(1-\alpha)$, the expected share exceeds p_B by at most $\sqrt{\mathbb{E}[c]\ln 2/(2k)}\leq 0.005,0.008,0.017$ for $k=2000$. Without the random order, owning the

last $c = 4$ positions would give $1 - (1 - \alpha)^{16} = 0.997$ at $\alpha = 0.3$; with it, π_{cap} still exceeds the 0.348 of Corollary 10, as one adversarial last revealer yields two candidate schedules. With $L_t = 25$ and $\ell_s = \ell_0 = 52$, $\delta_{\mathsf{LE}} \le 2^{-41.4}$ at $\alpha = 0.3$ without denial of service and $\delta_{\mathsf{LE}} \le 2^{-40.7}$ with $B = 1$ ($w_{\max} \le 0.01$, $p_1 = 0.514$).

7.4 From the Ideal Model to the Protocol, Against Quantum Adversaries

Lemma 14 (Reprogramming hidden inputs [43])**.** *Let a quantum process make q queries to an oracle O, and let its run with O be determined by randomness independent of a random variable Z, so that, with that randomness fixed, its state before each query does not depend on Z. Let S_Z be a set of inputs, determined by Z, with $\Pr_Z[x \in S_Z] \le p$ for every input x, and let O' agree with O outside S_Z. Then the final states of the process with O and with O' have expected trace distance at most $2q\sqrt{p}$.*

Proof sketch. Each query changes the state by at most $2\|\Pi_{S_Z}|\phi_i\rangle\|$, of expectation at most $\sqrt{p}$ as $|\phi_i\rangle$ is independent of Z (Appendix S4). □

Theorem 15 (Ideal to real, QROM)**.** *Model the commitment and seed functions of QPoS-S, cSHAKE256 with distinct tags, as a quantum random oracle independent of the score oracle ρ and of ML-DSA's internal hashing. Let commitments, secrets and seeds have n_c, n_r and n_s bits, and let identities and epochs be fixed-length fields of n_p bits in total. Assume mildly adaptive corruption (Section 4), the conditions of Lemma 12, that* Fail *does not occur, and that $c \le \bar{c}$; on $\neg$Fail, $\bar{c} = L_t - 1$ suffices. Let $m = 2^{\bar{c}}$, and let t count all oracle queries of all parties before τ^*. Then there is an m-ideal experiment such that, for every $[0,1]$-valued function F of the epoch's leaders and the adversary's final output,*

$$\left|\mathbb{E}_{\mathsf{real}}[F] - \mathbb{E}_{\mathsf{ideal}}[F]\right| \le \epsilon_{\mathsf{lead}} = \frac{2^{n_p+2n_r}}{2^{n_c+1}} + \frac{2t}{2^{n_r/2}} + \frac{2t\sqrt{m}}{2^{n_s/2}} + \frac{m^2}{2^{n_s+1}}.$$

Queries made after τ^* do not appear in the bound.

Proof sketch. Four hybrids give the four terms: (1) statistical binding of the wide commitments leaves late adversaries only reveal-or-withhold; (2) Lemma 14 with $Z = r_{i^*}$ hides r_{i^*} before τ^*; (3) Lemma 14 on the pre-sampled seeds Z_b of the at most m reveal sets makes their scores and order keys fresh; (4) the Z_b are distinct except with probability $m^2/2^{n_s+1}$, giving an m-ideal experiment. Before τ^*, validator i^* uses r_{i^*} only through its commitment, which is uniform for every r_{i^*}, so the run before τ^* is independent of the reprogrammed inputs; the run after τ^* is the same map in all hybrids, so the trace distance at τ^* bounds the final distance and later queries are free (Appendix S4). □

QPoS-S uses $n_r = n_s = 512$, $n_c = 1536$ and 64-bit identity and epoch fields ($n_p = 128$). For $t = 2^{64}$ and $\bar{c} = L_t - 1 = 24$, the four terms are 2^{-385}, 2^{-191}, 2^{-179} and 2^{-465}, so $\epsilon_{\mathsf{lead}} \le 2^{-178.9}$; $\bar{c} = 24$ is a worst case. Scores and order keys use H with $h = 256$.

Corollary 16 (Real protocol)**.** *Let $m = 2^{L_t - 1}$ and $\epsilon_{\mathsf{LE}} = \epsilon_{\mathsf{lead}} + \epsilon_{\mathsf{disc}}$, where $\epsilon_{\mathsf{disc}} = (k+1)mN(N-1)2^{-h}$ applies Lemma 8 to the k slot races and the order race of each candidate, and N counts all registered identities. Against quantum adversaries with mildly adaptive corruption, under the conditions of Lemma 12, every bound of Theorem 11, Proposition 13, and Lemma 12 holds in the real protocol, for the first E epochs, up to an additive $2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$. In particular, the adversary proposes in a fixed target slot with probability at most $\pi_{\mathsf{cap}} + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$, and an epoch has no guaranteed slot among n consecutive slots placed by the adversary with probability at most $(k - n + 1)\mu p_B^n + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$.*

Proof sketch. Idealizing epochs one at a time costs $\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}}$ each, plus $\Pr[\neg \mathsf{OK}_E] \le E\delta_{\mathsf{LE}}$ (Appendix S4). □

Consequences. By Corollary 16, an honest transaction is included within n slots of any adversarially chosen start, except with probability $\zeta + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$, whenever $n \ge \ln(k\mu/\zeta)/(-\ln p_B)$, as used by Proposition 4. For $\alpha = 0.3$, $k = 2000$ and $\zeta = 2^{-40}$, $n = 31$ suffices without denial of service and $n = 56$ with $B = 1$; for $E = 2^{20}$, $2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}}) < 2^{-19}$ at $\alpha = 0.3$ and $< 2^{-36}$ at $\alpha = 0.2$, with or without $B = 1$. Finality additionally needs the finality gadget's liveness, and a window that straddles an epoch boundary adds the bounds of both epochs. No PQ-Vault role is exclusive to a slot: anyone can aggregate certificates, and attestors, not leaders, certify checkpoints.

Long-range attacks. With Vault as the weak-subjectivity source, a fork before the latest anchored checkpoint is rejected under Assumption A1, and a fork after it needs a certificate for the still-bonded attestor set, which Theorems 5 and 19 bound (Appendix S4).

8 Accepting Certificates on L1

Direct L1 verification ($\Theta(k_{\mathsf{rev}})$ ML-DSA verifications on calldata) costs 1.8×10^8 to 1.1×10^9 gas, above the per-transaction cap (Section 9.5), so certificates must be compressed.

8.1 A Non-PQ SNARK Cannot Be the Acceptance Path

Our earlier design [18] used a Groth16 [23] proof over certificates; this is unsound once L1 *accepts* checkpoints on its strength.

Proposition 17. *If* Vault *finalizes a checkpoint whenever a Groth16 proof Π of "all certificates verify" is valid, then a QPT adversary can finalize an arbitrary checkpoint, and with it arbitrary withdrawals, with probability $1 - \mathsf{negl}$.*

Proof sketch. Shor's algorithm recovers the CRS trapdoor, which lets the simulator prove false statements. The full proof is in Appendix S5. □

We therefore use an optimistic protocol with a PQ dispute path, whose happy path costs almost nothing; a transparent hash-based proof system is the alternative.

8.2 Optimistic Acceptance with Single-Reveal Fraud Proofs

Apart from the range check on W_{s} (done at proposal), Vf (Algorithm 1) is a *conjunction of per-coin checks*. An L1 attestor leaf holds (η_i, w_i), where η_i is the `EXTCODEHASH` of the contract storing i's expanded ML-DSA key (Section 9.5). For each coin index j, the *record* $r_j = (j, i_j, \eta_{i_j}, w_{i_j}, \sigma_{i_j}, L_{i_j}, \pi_j^{\mathsf{sig}}, \pi_j^{\mathsf{att}})$ holds the covering leaf and its two paths, and $\mathsf{Local}(j, r_j) = 1$ iff (i) $i_j < N$, $w_{i_j} > 0$ and $L_{i_j} \le c_j < L_{i_j} + w_{i_j} \le W_{\mathsf{s}}$, with c_j computed from the proposal's M_e, R_{sig}, W_{s} and its snapshot of R_{att}, W_{p} and N; (ii) π_j^{sig} and π_j^{att} have length d and open (σ_{i_j}, L_{i_j}) under R_{sig} and (η_{i_j}, w_{i_j}) under R_{att}, both at position i_j; and (iii) σ_{i_j} has the scheme's length and

verifies on M_e under the key stored in the contract with code hash η_{i_j}. The relayer commits to the records in a Merkle tree C_{cert} of depth $\lceil \log_2 k_{\mathsf{rev}} \rceil$, with r_j at index j (leaf encoding in Appendix S5).

Lemma 18 (Local decomposition)**.** *Except for a collision of* H, $\mathsf{Local}(j, r_j) = 1$ *for all* $j < k_{\mathsf{rev}}$ *if and only if the distinct positions* $P = \{i_j\}$*, their leaves, and the multiproofs assembled from the single paths form a certificate that Algorithm 1 accepts, instantiated with the vault's leaf encodings.*

Proof sketch. Without collisions, records naming one position carry equal leaves, and the checks of Algorithm 1 are exactly (i)–(iii) over all j. The full proof is in Appendix S5. □

Protocol. propose (Section 5.1) creates a bonded proposal at τ_0, with records in blobs (not relied on for safety). Until the *challenge deadline* $\tau_0 + T_{\mathsf{ch}}$ (extended to $\tau_{\mathsf{post}} + \Delta_{\mathsf{post}}$ after an open-all posting completed at τ_{post}), anyone may challenge(j) each index once, with bond b_C. A respond within Δ_{resp} carries one record: if it is unauthenticated under C_{cert} or names a wrong key address, the call reverts; otherwise it rejects the proposal if Local fails and closes the challenge if not. challengeAll (once, no bond) forces all records onto calldata within Δ_{open}; timeout and timeoutAll reject on an unanswered challenge or incomplete posting. finalize, with no open challenge or pending posting, finalizes if the epoch is $e^* + 1$, else marks it *stale*. Guards are $<$ the deadline for challenges and open-all, $\leq$ for responses and postings, $\geq$ for finalize and $>$ for timeouts, so no timestamp admits both an action and the transition its deadline unlocks. Bonds and rewards (b_C, b_R) and the full state machine are in Appendix S5.

Timing. With Δ_{chk} bounding a watcher's time to check all records and Δ_{prep} the relayer's time to answer, we require

$$T_{\mathsf{ch}} > \Delta_{\mathsf{chk}} + \Delta_{\mathsf{inc}}, \qquad \Delta_{\mathsf{post}} > \Delta_{\mathsf{chk}} + \Delta_{\mathsf{inc}},$$

and $\Delta_{\mathsf{open}}, \Delta_{\mathsf{resp}} \geq \Delta_{\mathsf{prep}} + \Delta_{\mathsf{inc}}$. Each pending proposal is then eligible for a final transition by $T_{\mathsf{fin}} = T_{\mathsf{ch}} + \Delta_{\mathsf{open}} + \Delta_{\mathsf{post}} + \Delta_{\mathsf{resp}}$ after its inclusion. With $T_{\mathsf{ch}} = 7$ days, $\Delta_{\mathsf{open}} = \Delta_{\mathsf{resp}} = 1$ day and $\Delta_{\mathsf{post}} = 2$ days (as in optimistic rollups [26]), $T_{\mathsf{fin}} \leq 11$ days.

Theorem 19 (Optimistic acceptance)**.** *Assume Assumptions A1 to A4 and the timing conditions above. Then the following hold. (Safety) The probability that* Vault *finalizes a proposal whose* M_e *meets the winning condition of Definition 1, with the attestor list committed by the proposal's snapshot of* R_{att}*, is at most the bound of Theorem 5 plus* $\delta_{\mathsf{col}} = (12t^3 + 2q_C)/2^{h_C}$*, where* $q_C = 2k_{\mathsf{rev}}(\lceil \log_2 k_{\mathsf{rev}} \rceil + 2d + 7)$ *and* h_C *is the smallest output width of* C_{cert}*,* R_{sig} *and* R_{att} *(*$h_C = 256$ *in the deployed vault). Since* Col *ranges over the whole oracle,* δ_{col} *also excludes two key contracts with the same code hash* η*, and, in Theorem 20, two headers with the same* M_e *or two withdrawal trees with the same root. (Liveness) An honest relayer's proposal is never rejected except with probability* δ_{col}*; it is eligible for finalization at* $\tau_0 + T_{\mathsf{ch}}$ *if no dispute is raised, and by* $\tau_0 + T_{\mathsf{fin}}$ *otherwise; if the relayer then submits* finalize*, it is finalized within a further* Δ_{inc}*. (Cost) Each frivolous coin challenge costs its sender* b_C*, and an honest watcher spends at most one open-all transaction fee and one bond* b_C *per proposal; if its challenge is the first on a failing record, the bond is returned with* b_R*.*

Proof sketch. Excluding collisions (Lemma 1), by Assumption A4 and the strict timing conditions, an honest watcher learns all committed records (from blobs or via open-all) and challenges any failing record before the deadline; the only non-reverting response rejects the proposal, and an unanswered challenge rejects it at timeout. A finalized proposal thus has only passing records, an accepted certificate by Lemma 18, bounded by Theorem 5. Liveness: honest records pass and answers meet the inclusive deadlines. The full proof is in Appendix S5. □

8.3 End-to-End Custody

Theorem 20 (Custody safety)**.** *Assume Assumptions A1 to A4, the timing conditions above, and, in every epoch* e*, corrupt attestor stake below* $W^e/3$ *with* $W^e_{\mathsf{p}} = \lfloor W^e/3 \rfloor$*. Assume also the honest-signing rule of Section 5, and let* Fin *be the event that two honest validators consider conflicting checkpoints of the same epoch final. Then over* E *epochs, for every QPT adversary making* t *oracle queries in total,*

$$\Pr[\textit{G1 violated}] \leq E\Big(12t^2\varepsilon^* + \frac{48t^3 + 2q_V}{2^h} + \delta_{\mathsf{col}}\Big) + 2E\mathsf{Adv}^{\mathsf{uf}}_{N_{\mathsf{tot}}} + \mathsf{Adv}^{\mathsf{uf}}_{U} + \Pr[\mathsf{Fin}],$$

where $\varepsilon^* \leq 2^{-\lambda}/(12t^2)$ *under Eq.* (5) *when* $t \leq q(1 + 2^{-20})$ *(Section 6.4),* N_{tot} *is the number of honest attestor keys generated over the whole experiment, and* U *is the number of user keys.*

Proof sketch. By induction over checkpoints we keep the invariant I_e: finalized $M_1, \ldots, M_e$ are honest, the vault's (R_{att}, W, N) is the honest one of epoch $e + 1$, and each payment pays a leaf of a finalized $\omega_{e'}$ at most once. A proposal for epoch e uses the honest snapshot by I_{e-1}; if no honest attestor signed M_e, Theorem 19 bounds its finalization, and otherwise the honest-signing rule makes M_e honest outside Fin. Nullifiers, balance conservation and user-signature unforgeability bound payments by the escrow. The full proof is in Appendix S5. □

The union bound over epochs is why the reveal rule needs $\lambda + \log_2 E$ for a lifetime target (Section 6.4); Fin is the safety of the sidechain's finality gadget, which we do not re-analyze (Appendix S5).

Liveness (G4). If honest transactions are included on the sidechain within Δ_{sc} (except with probability $\zeta + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$, Corollary 16) and some honest party relays each checkpoint, an honest withdrawal is paid within $\Delta_{\mathsf{sc}} + \Delta_{\mathsf{cp}} + \Delta_{\mathsf{inc}}$, where Δ_{cp} includes at most $2(T_{\mathsf{fin}} + \Delta_{\mathsf{inc}})$ for finalization, including the predecessor's wait when the checkpoint interval is at least $T_{\mathsf{fin}} + \Delta_{\mathsf{inc}}$ (Section 5.4, Theorem 19 and Assumption A1), if an honest relayer calls finalize once eligible.

9 Implementation and Evaluation

Artifacts and setup. `pqcert` implements Algorithm 1 in Rust with the QROM rule (Eq. (5)) and Keccak-256 Merkle multiproofs, over the round-3 Dilithium implementation used by the prototype of [38] (labeled Dilithium; not interoperable with FIPS 204, which a second backend cross-checks). We also built a Python verifier, a network harness, Monte Carlo simulators and the Solidity vault of Section 9.5, all public [21] (Appendix S6). The verifier rejected all negative tests in every run. Timings are single-threaded medians on a

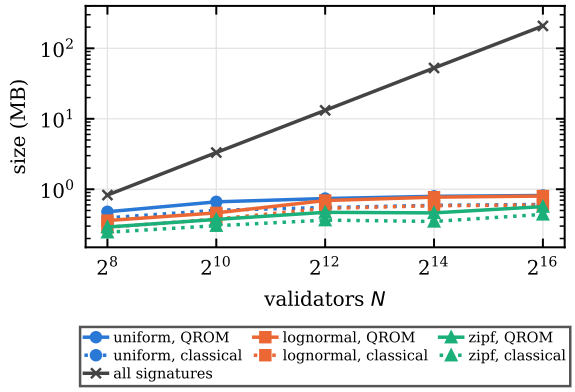


Fig. 2. Certificate size vs. number of validators (round-3 Dilithium2, $W_p = \lfloor 0.3333\,W \rfloor$, 85% signed). Colors: stake distribution; solid: QROM rule; dotted: classical rule; black: all signatures.

2-vCPU cloud VM (*M1*). Unless stated otherwise, $\lambda = 128$, $\log_2 q = 64$, $\kappa_Q = 4$, $\delta = 0.1$, $W_p = \lfloor W/3 \rfloor$ (equivalently $\lfloor 0.3333\,W \rfloor$ in the size runs, Appendix S6), stake is uniform, log-normal ($\sigma = 1.5$) or Zipf ($s = 1$), and validators sign in random order until at least 85% of the stake has signed (under Zipf, up to 99% at $N = 2^8$).

9.1 Certificate Size and Cost

Figure 2 and Table 5 report certificates with real round-3 Dilithium2 signatures. *(1) Compactness.* At fixed λ, q and W_s/W_p, k_{rev} does not depend on N, and the size $O(k_{rev}(|pk| + |\sigma|) + hk_{rev} \log N)$ grows logarithmically in N while W_s/W_p stays bounded away from 1: from 0.36 MB at $N = 2^8$ to 0.80 MB at $N = 2^{16}$ (log-normal, QROM). Shipping all signatures costs 208 MB at $N = 2^{16}$, 260× more, a ratio growing as $\Theta(N/\log N)$. *(2) Cost of quantum security.* The QROM rule raises k_{rev} by 35% (143 to 193), but coins collide on heavy attestors, so distinct reveals grow less: the size overhead is 19–34% across all configurations (23–34% for log-normal stake), and verification time grows by 20–58% for log-normal stake. *(3) Stake skew helps.* Under Zipf stake, coins concentrate on large validators and the certificate shrinks to 0.57 MB at $N = 2^{16}$. At $N = 2^{16}$, verification takes 18 ms on one core and building 0.47 s, dominated by hashing the signature tree. As Table 4 predicts, the proven threshold dominates cost: at $N = 4096$, raising W_p from $W/3$ to $W/2$ and $2W/3$ grows the certificate from 0.69 to 1.06 and 1.95 MB. Table 6 reports 384-bit commitments (Section 6); its totals also reflect slightly different reveal sets and a separate timing run, and with the reveal set fixed the wider hashes alone add 3.1–5.7%.

FIPS 204 cross-check and light clients. On a third machine, *M3* (2 vCPUs, Intel Xeon, 2.1 GHz), we ran the harness with round-3 Dilithium2 and with FIPS 204 ML-DSA-44 for $N = 2^8$–2^{16} (log-normal, $W_p = \lfloor W/3 \rfloor$). k_{rev} is identical; sizes differ by −9.5% to +2.3% (the latter for the classical rule at $N = 2^{16}$) with no consistent sign, because different signatures change which positions the coins select; FIPS 204 verification is 2–33% slower. At $N = 2^{16}$ (QROM), the certificates are 0.80 and 0.81 MB and verify in 18.4 and 21.1 ms. The Python verifier (FIPS 204) checks a QROM certificate at $N = 4096$ in 25, 44 and 57 ms on M1 for ML-DSA-44/65/87, and in 18, 27 and 44 ms on a laptop VM (*M2*). L1, not light clients, is the bottleneck (Section 9.5).

TABLE 5
Certificates on M1 (round-3 Dilithium2, log-normal stake, QROM rule). "Distinct": distinct reveals; "Gain": size of all signatures divided by certificate size.

N	k_{rev}	Distinct	Size (MB)	Gain	Build (ms)	Verify (ms)
2^8	192	94	0.36	2.3×	2.4	8.7
2^{10}	192	118	0.46	7.2×	7.9	11.3
2^{12}	193	173	0.69	19×	29.4	16.3
2^{14}	193	188	0.77	68×	129	21.6
2^{16}	193	189	0.80	260×	474	18.4
Classical rule, 2^{16}: $k_{rev} = 143$, 140 distinct, 0.60 MB, verify 13.7 ms						

TABLE 6
Keccak-384 vs. Keccak-256 commitments (round-3 Dilithium2, log-normal, QROM, M1).

	Certificate size (MB)			Verify (ms)	
N	$h = 256$	$h = 384$	Increase	$h = 256$	$h = 384$
2^{12}	0.692	0.734	+6.1%	19.0	20.4
2^{14}	0.772	0.801	+3.8%	20.6	21.7
2^{16}	0.801	0.857	+7.0%	21.2	20.7

9.2 Baselines: Falcon and Hash-Based Signatures

The signature scheme sets most of the cost. Table 7 compares the Dilithium levels with Falcon-512 and with the synchronized hash-based signatures (generalized XMSS with target-sum encoding) that Drake *et al.* [6] propose for post-quantum Ethereum, measured with their implementation [44] at lifetime 2^{32}. The hash-based rows combine their measured size and verification time with the reveal structure of the Dilithium2 (r3) certificate (173 distinct reveals, 1,328 multiproof hashes), as coin selection is scheme-independent; the other rows are complete `pqcert` certificates.

Compact certificates beat shipping all signatures by 18–20× at $N = 4096$. Falcon-512 is the best fit: its certificate is 2.2× smaller than with Dilithium2 (r3) (0.40 MB at $N = 2^{16}$, verified in 11 ms), at the price of floating-point signing. The hash-based signatures of [6] give smaller certificates than ML-DSA thanks to 52-byte keys, but verify 2.5–4.2× slower here and are *synchronized*: a key signs at most once per epoch. Their hash-based succinct aggregation, which we did not measure, gives constant-size proofs but needs a SNARK prover. Compact certificates lose for very small sets (below about 2^6 signers, extrapolating Table 5) and at high thresholds such as $W_p = 2W/3$ (Table 4). On L1 the verifier contract dominates every per-reveal cost: Falcon verification is reported at 1.5–3.9M gas [12], versus 1.3–8.2M for an ML-DSA-44 dispute response here (Table 8), which affects only the dispute path.

9.3 Validating the Grinding Term

The attack of Theorem 6 relies on each fresh root independently covering all coins with probability $\beta^{k_{rev}} \geq (w(C)/W_s)^{k_{rev}}$, where $w(C)$ is the validly covered weight. With a SHA3-256 coin function of the same form, an adversary covering $w(C)/W_s \in \{1/2, 1/3\}$ re-randomizes its root

TABLE 7
Signature-scheme baselines on M1 ($N = 4096$, log-normal, QROM, $3{,}524$ signers). "All sigs": size of shipping every signature and key. Rows marked (r3) use round-3 Dilithium (Section 9). *Composed from measured components (see text).

Scheme	pk / sig (B)	sign / verify (ms)	Cert (MB)	Cert verify (ms)	All sigs (MB)
Dilithium2 (r3)	1312 / 2420	0.42 / 0.076	0.69	16.3	13.2
Dilithium3 (r3)	1952 / 3293	0.75 / 0.12	0.94	26.4	18.5
Dilithium5 (r3)	2592 / 4595	0.74 / 0.22	1.27	41.4	25.3
Falcon-512	897 / 666	0.20 / 0.036	0.31	8.7	5.5
XMSS-TS, hashing-opt. [6]	52 / 3116	0.77 / 0.22	0.59*	40*	11.2
XMSS-TS, size-opt. [6]	52 / 2092	1.55 / 0.38	0.42*	69*	7.6

until all k_{rev} coins are covered. Over 120–300 runs per point, the mean number of trials matches $(W_{\mathsf{s}}/w(C))^{k_{\mathsf{rev}}}$ within two standard errors for $k_{\mathsf{rev}} = 2$–12 and $k_{\mathsf{rev}} = 2$–8, respectively (e.g., $4{,}252$ trials at $k_{\mathsf{rev}} = 12$ against $4{,}096$ predicted). Figure 3(b) applies the attack to classically parameterized certificates: it caps their security at about 81 bits for $q = 2^{64}$ and 65 bits for $q = 2^{80}$, against the intended 128, whereas under the QROM rule Theorem 5 guarantees λ bits for the grinding term.

9.4 Networked Certification

An aggregator collects real FIPS 204 ML-DSA-44 signatures over TCP from validators in two worker processes, builds the certificate and sends it to 8 light clients, under emulated WAN conditions (five regions at 1–95 ms one-way latency, 10% of validators offline, collection until 80% of the stake; full setup in Appendix S6). Over five rounds, the median end-to-end time, from header broadcast until all light clients have verified, is 321, 288, 408 and 647 ms for $N = 128$, 512, 1024 and 2048, and no round exceeded 716 ms. Beyond the round trip to the farthest needed region, collection time grows with sequential signature verification at the aggregator (0.13 ms per signature, 221 ms for the $\approx$1,720 signatures at $N = 2048$), which parallelizes trivially, and construction takes at most 22 ms. Certification is far from limiting withdrawal latency, which T_{ch} dominates.

9.5 Cost on Ethereum: Measured Contracts

We implemented the vault and dispute state machine of Sections 5 and 8 in Solidity (`PQVault.sol`, about 300 lines) on the ZKNOX ML-DSA verifiers [11], and measured transaction-level gas (EIP-7623 [45]) with Foundry for the NIST-conformant and the packed ZKNOX verifier (Table 8). The test vectors are a real certificate over $N = 2^{16}$ positions with 872 FIPS 204 signatures, which both verifiers accept; 17 tests per verifier confirm the state machine (Appendix S6).

The optimistic path (propose plus finalize) costs about 4.2×10^5 gas per checkpoint (4.1×10^5 in the happy-path test). Answering a coin challenge costs 8.2×10^6 (NIST) or 1.3×10^6 gas (packed; 1.4×10^6 with the challenge), paid by the party in the wrong. Direct verification costs $2{,}600\times$ and $430\times$ the optimistic path, and its 0.81 MB of calldata exceeds the common 128 KB mempool limit; extrapolated to the 189 distinct reveals at $N = 2^{16}$ (Table 5), it would cost 1.6×10^9 and 2.6×10^8 gas (Appendix S6). Even the draft EIP-8051 precompile [5] would leave direct verification bound by calldata (20,512-byte expanded keys). An open-all posting (803,580 bytes in 7 transactions) costs 3.0×10^7 gas, set almost entirely by the calldata floor; execution adds only 1.6–2.0×10^6.

TABLE 8
Measured L1 gas of `PQVault` (Foundry; $k_{\mathsf{rev}} = 193, 130$ distinct reveals, $d = 16$). Storage written earlier in a test is warm. All operations fit in one transaction except direct verification, which exceeds the 2^{24}-gas cap [46] (parentheses: minimum number of transactions), and the open-all posting, split for the 128 KB mempool limit. Details in Appendix S6.

Operation	Gas, NIST verifier	Gas, packed verifier
Register key (once, 22.4 KB ZKNOX format)	6.8×10^6	6.8×10^6
Deposit	3.5×10^4	3.5×10^4
Propose checkpoint (computes M_e, checks k_{rev})	3.3×10^5	3.3×10^5
Finalize checkpoint	9.0×10^4	8.9×10^4
Withdraw (depth-20 proof)	1.0×10^5	1.0×10^5
Coin challenge	5.3×10^4	5.4×10^4
Respond (one ML-DSA verification)	8.2×10^6	1.3×10^6
Open-all request	2.4×10^4	2.4×10^4
Post all 193 records (7 txs, 0.80 MB)	3.0×10^7	3.0×10^7
Direct verification, all 193 coins	1.1×10^9 (≥ 65 txs)	1.8×10^8 (≥ 11 txs)

9.6 Leader Election

Fairness and Sybil invariance. We evaluated the score of Eq. (3) with SHA3-256 over a per-validator value (a hash of its index), the slot and a fixed seed, for 200 validators with log-normal stake and $40{,}000$ slots. Win frequencies match stake fractions ($\chi^2 = 190.8$ with 199 degrees of freedom, $p = 0.65$). Sampling the race of Lemma 8, an adversary with $\alpha = 0.2$ split into $m \in \{1, 2, 5, 10, 50\}$ identities over 2×10^5 slots keeps a share between 0.198 and 0.203 (standard error 0.0009), as Lemma 8 predicts.

Steering. The remaining experiments sample the exponential-race model of Lemma 8 directly; they check our derivations rather than an implementation. Figure 3(c) compares target-slot capture. Under the runner-up rule, the simulation (adversary sees both candidate seeds) matches case (a) of Corollary 10, 0.347 vs. 0.348 at $\alpha = 0.3$, and 4 identities raise it to 0.357, so Sybil invariance fails. With a fixed QPoS-S seed the probability equals α within sampling error; with seeds steered across epochs it stays below π_{cap} (Proposition 13), within sampling error at $\alpha = 0.05$, where the bound is nearly tight. Choosing among 2^c epoch seeds ($k = 2000$) gains 0.004–0.038 in epoch share for $c \leq 12$ (an illustrative range; Proposition 13 makes $c \geq 4$ rare), always below the bound $\sqrt{c \ln 2/(2k)}$ of Theorem 11 (0.013–0.046).

Reveal order and fallback proposers. A further experiment (`reveal_order_sim.py`) checks Proposition 13 and Theorem 11, with 300 honest validators of log-normal stake in its first two parts. More adversarial identities raise $\Pr[c \geq x]$ towards, but not above, α^x: at $\alpha = 0.3$, 100 identities give 0.299, 0.088, 0.026 and 0.0077 for $x = 1, \ldots, 4$ (10^5 orders each), against 0.3, 0.09, 0.027 and 0.0081; a single identity never has $c \geq 2$. Over 10^5 consecutive epochs in which the adversary (20 identities) always picks the candidate maximizing the next c, $\mathbb{E}[2^c]$ is 1.14, 1.42 and 2.19 for $\alpha = 0.1$, 0.2 and 0.3 (bound μ: 1.14, 1.5 and 4), and c never reached $L_t = 25$; its target-slot capture rate, 0.112, 0.255 and 0.442, is an example rather than a worst case (bound π_{cap}:

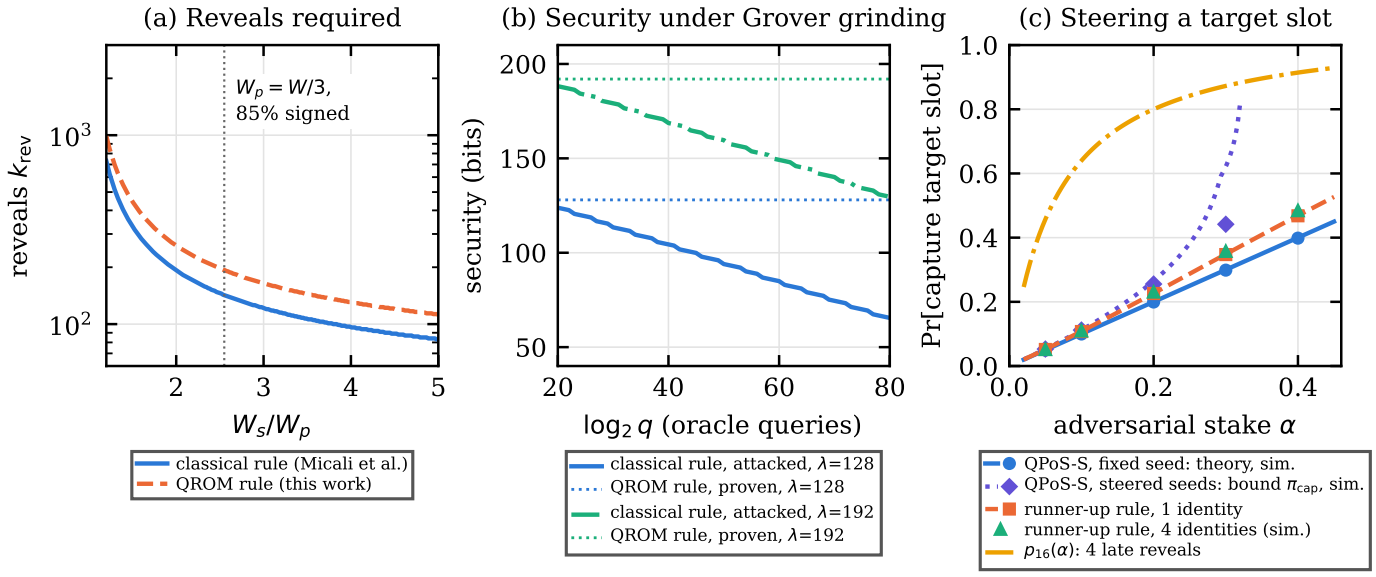


Fig. 3. (a) Reveals required by the classical and QROM rules. (b) Security of *classically* parameterized certificates under the attack of Theorem 6 ($W_s/W_p = 2.55$, $N = 2^{16}$); dotted: level λ guaranteed by Theorem 5 under the QROM rule. (c) Target-slot capture under QPoS-S, with a fixed seed and with seeds steered across epochs (dotted: bound π_{cap} of Proposition 13 for $B = 0$; diamonds: the simulated adversary of Section 9.6, which maximizes c rather than capture), and under the runner-up rule (lines: theory; other markers: 4×10^5 simulated trials each).

0.112, 0.265 and 0.622). The third part (30 adversarial and 70 equal-stake honest identities, 2×10^5 slots) finds slots not guaranteed with frequency 0.300 and 0.511 at $\alpha = 0.3$ for $B = 0$ and $B = 1$, against $p_B = 0.300$ and 0.514.

10 Discussion and Conclusion

Scope of Assumption A1. A CRQC that breaks ECDSA also breaks Ethereum's BLS consensus signatures, so Assumption A1 is not implied by our threat model: it assumes that L1 consensus is not attacked, or is restored by a state-preserving hard fork [4]. Assumption A3 is needed only for transactions that expose an ECDSA key (deposits, initial bindings, bonds of relayers and watchers); PQ smart accounts would remove it (Appendix S7).

Data availability and watchers. Theorem 19 assumes one honest online watcher; withholding blob data cannot cause a false acceptance (Section 8). For a gas-price ceiling $p^{\max}$, $b_C \geq 8.2 \times 10^6 p^{\max}$ covers a response and $b_R \geq b_C$ plus a reward. Open-all requests carry no bond, so a watcher spends at most one request fee and one bond b_C per proposal, while relayers must recover the 3.0×10^7-gas posting through fees, on which G4 relies. Any depositor can watch at about 20 ms per checkpoint (Appendix S7).

Public leader schedule. The public schedule enables Theorem 15 but exposes leaders and lets late revealers see candidate schedules. With $L = 3$ fallback proposers, an adversary suppressing B honest proposers per slot affects only slots that are not guaranteed, so inclusion within 31 to 56 slots holds with overwhelming probability at $\alpha = 0.3$ (Theorem 11 and Corollary 16). The stake-weighted reveal order makes late reveals rare ($\Pr[c \geq x] \leq \mu\alpha^x$, Proposition 13); target-slot capture is at most 0.622 at $\alpha = 0.3$ (runner-up attack: 0.348; owning the last four reveals: 0.997) and at most 0.265 for $\alpha \leq 0.2$. Instant adaptive corruption, which could seize the last honest revealer, is outside our model, and hidden schedules via a PQ VRF or secret leader election are left open (Appendix S7).

Limitations. The sidechain is analyzed by simulation only, the network experiment emulates WAN latency on shared cores, and gas was measured in a local Foundry EVM. If the corrupt third abstains and 85% of honest stake signs, $W_s/W_p = 1.7$ and $k_{rev} = 340$, which enlarges certificates but leaves the optimistic L1 path unchanged. Our leader-election bounds assume a standard BFT finality gadget, which we did not re-analyze, that finalizes each block of the reveal window within its slot. The prototype does not implement the binding registry or the canary. Any EUF-CMA scheme fits; Falcon-512 halves certificate size but is hard to sign in constant time (Appendix S7).

Conclusion. Protecting Ethereum assets against quantum adversaries without changing L1 is possible only for escrowed assets, and only if every predicate on the release path is PQ. PQ-Vault realizes this with quantum-secure compact certificates, steering-resistant stake-weighted PoS and single-reveal fraud proofs. These guarantees cost 19–34% more certificate size, certification in 0.65 s at 2048 validators, and about 4.2×10^5 gas per checkpoint on L1.

Appendices. The appendices contain the full proofs of all results proved in the paper, the complete specification of the dispute protocol, and additional details and discussion. Appendix Sx supplements Section $x + 3$; each result is restated before its proof.

S1 System and Threat Model: Proofs

S1.1 On Assumption A1

Assumption A1 is unavoidable: if Ethereum's own consensus is subverted, no contract on Ethereum can protect anything. Our goal is to protect assets against the failure of *account-level* cryptography (Section 10).

S1.2 Proofs of Proposition 2 and Corollary 3

Proposition 2 (Custody impossibility). *Let* Π *be any protocol that does not modify L1 validity rules. Suppose that at some time* $t \geq \tau_Q$ *an asset* a *is still spendable under the L1 predicate* $\mathsf{ECDSA.Vf}(pk_M, \cdot) = 1$ *with* pk_M *public. Then a QPT adversary can produce an L1-valid transaction spending* a *that L1 cannot distinguish from one signed by the owner. Nothing produced by* Π *can make this transaction invalid.*

Proof. $\mathcal{A}$ runs Shor's algorithm on pk_M to obtain sk_M and signs a transaction that transfers a. L1 validity depends only on the predicate and on the account state, and sk_M produces exactly the signatures the owner would produce. Since Π does not change L1 validity rules, no message, proof, or state of Π can invalidate the transaction. Whether the adversary or the owner moves a first is then decided by transaction ordering alone, which the protocol does not control. □

Corollary 3. *Consider, on unmodified L1, an asset that must remain* live*, i.e., movable by its owner, and* safe *against* $\mathcal{A}$ *after* τ_Q*. Every predicate that can release it after* τ_Q *must then rest on PQ assumptions and on Assumption A1 alone; in particular, no ECDSA key whose public key is exposed may authorize its release, or change or redirect such a predicate without a PQ-authorized veto. On an account-based L1 this requires holding the asset under a contract predicate* Φ*.*

Proof. An ECDSA key that can release the asset, directly or by changing the predicate without a veto, falls under Proposition 2 once its public key is exposed. An EOA is controlled by its ECDSA key, so only contract code can implement a different predicate. □

Proposition 2 rules out designs in which validators "check a PQ proof before including" an L1 transaction. The sidechain design of [18] implicitly assumed such a check.

The corollary does not cover addresses whose public key has never been exposed. Attacking their address commitment is a different problem, and their first spend exposes the key (Assumption A3). Nor does it count an asset made permanently unspendable as protected: that would be safe but not live.

S2 PQ-Vault Design: Details and Proof

S2.1 Why the Earlier Zero-Knowledge Binding Is Replaced

The sidechain design of [18] bound pk_M to pk_S with a zero-knowledge proof that $pk_M = \mathsf{KGen}(F(Seed))$. That proof requires secp256k1 scalar multiplication inside a transparent circuit, which is costly. It also does not help after τ_Q: the adversary learns sk_M, and the L1 predicate that matters still accepts sk_M (Proposition 2). We replace it with a binding that is authenticated *classically, while classical authentication is still meaningful*, and that becomes immutable afterwards.

S2.2 Binding Mechanism in Full

The binding has three parts. In the *bind* step, $Addr_M$ calls $\mathsf{bind}(\mathsf{H}(pk_S))$; if no binding exists, it takes effect immediately, and the L1 transaction sender authenticates $Addr_M$. A later *rebind* from $Addr_M$ creates a pending rebinding with a fresh per-address counter n and activation time $\tau_{\mathsf{act}} = \tau_{\mathsf{req}} + D$. At most one rebinding per address may be pending, so the owner never has to cancel more than one at a time; applyRebind succeeds only in a block with timestamp strictly greater than τ_{act}, and only if counter n is not cancelled. Until then, the current PQ identity can *veto* it with an ML-DSA signature on $\mathsf{cancel}(\mathsf{Vault}, Addr_M, n)$, submitted to the sidechain. The cancellation reaches Vault through a certified checkpoint, whose binding-operations root β_e is part of M_e; no ML-DSA verification on L1 is needed. A finalized root does not by itself change the registry: anyone, normally the owner, then calls $\mathsf{applyCancel}(Addr_M, n, e, \pi)$, which checks that π opens the operation $(\mathsf{cancel}, Addr_M, n)$ under the finalized β_e and marks counter n cancelled. A cancellation is effective if this call is included at a timestamp at most τ_{act}; it consumes n, and an operation for any other counter has no effect.

Quantum canary. The canary makes the registry immutable once quantum attacks are demonstrated. Vault holds a bounty locked to an address $Addr^\star$ derived from a nothing-up-my-sleeve secp256k1 point whose discrete logarithm no one knows. The canary fires on a valid ECDSA signature for $Addr^\star$, checked with `ecrecover`, on the *fixed* digest $z^\star = \mathsf{H}(\text{CANARY} \,\|\, chainid \,\|\, \mathsf{Vault})$, and then permanently disables all ECDSA-authorized registry operations. The digest must be fixed: for a digest chosen by the submitter, a valid signature exists without the key (pick $u, v \neq 0$, let $R = uG + vQ$, $r = x(R)$, $s = rv^{-1}$ and $z = us$). A signature on $z^\star$ is a forgery of ECDSA on a predetermined message, which for a key with unknown discrete logarithm signals a cryptanalytic or quantum break.

S2.3 Further Details of the Vault and Sidechain

Vault. The sidechain credits each deposit event once, after it is final on L1. Several proposals for the same epoch may be pending, and a rejected epoch can be proposed again; the dispute functions are specified in Section 8. When finalize succeeds, other proposals for epoch e become stale. The sidechain assigns each withdrawal a globally unique $nonce$, so ξ is a nullifier: an authorized withdrawal that appears in several checkpoints is paid at most once. Recipients that are not fresh addresses or PQ smart accounts would expose the withdrawn funds again (Proposition 2).

Sidechain. H_n denotes n output bits of cSHAKE256 with a PQ-Vault customization string, so that its inputs are separated from Keccak-256 and from ML-DSA's internal SHAKE256. Honest validators send r_i to no one before their phase or sub-window starts. A validator whose reveal for epoch e is missing is suspended from epoch $e + 3$ on, the first epoch whose registrations are still open, until it registers again; it is not slashed, because a censored honest reveal looks the same. Every node computes the schedule of the whole epoch once the seed is final. Proposer eligibility needs no proof beyond the block signature. The 1536-bit commitments make each commitment open to at most one secret, statistically in the QROM, and the 512-bit secrets and seed keep the quantum hiding errors small (Theorem 15). They add 192 bytes to a registration that already carries a 2,420-byte signature.

S2.4 Proof of Proposition 4

Proposition 4 (Binding integrity). *Suppose* $\mathsf{Bind}[Addr_M] = \mathsf{H}(pk_S)$ *was set by* $Addr_M$*'s owner, and let* $\Delta_{\mathsf{v}} = \Delta_{\mathsf{obs}} + \Delta_{\mathsf{sc}} + \Delta_{\mathsf{cp}} + \Delta_{\mathsf{apply}}$*, where* Δ_{obs} *bounds the owner's time to observe a pending rebinding and sign,* Δ_{sc} *bounds sidechain inclusion of its transaction,* Δ_{cp} *bounds the time from inclusion to finalization of the carrying checkpoint, including disputes and the* finalize *call (Theorem 19), and* Δ_{apply} *bounds preparing and including* applyCancel*. Suppose* $D > \Delta_{\mathsf{v}}$*, the owner is online at least once in every window of length* $D - \Delta_{\mathsf{v}}$*, these bounds hold, and Assumptions A1 to A4 and the timing conditions of Theorem 19 hold. By Corollary 16 and Theorem 19,* Δ_{sc} *and* Δ_{cp} *fail only with the probabilities given there. Then a QPT adversary can change the binding only by forging an ML-DSA signature, by breaking certificate soundness, or before the owner's first binding transaction is included.*

Proof. Before the owner's first binding transaction is included, no binding exists, and this case is excluded by the statement. Afterwards, any change to $\mathsf{Bind}[Addr_M]$ goes through a pending rebinding that lasts D. Because the owner is online in every window of length $D - \Delta_{\mathsf{v}}$, it is online at some time at which at least Δ_{v} of the delay remains. It then signs a cancellation of counter n with sk_S, which is included on the sidechain, finalized on L1, and applied by applyCancel no later than τ_{act}. Since applyRebind needs a timestamp strictly greater than τ_{act}, the rebinding can never be applied. An adversary can prevent this outcome in only two ways. It can produce an ML-DSA-signed operation under pk_S that the owner never signed; operations signed for earlier counters have no effect on n, so this is a signature forgery. Or it can have the vault finalize a checkpoint that omits or reverses binding operations that are final on the honest sidechain. Such a checkpoint is not signed by any honest validator, and its finalization is bounded by Theorem 19 (with Theorem 5 on the direct path). After the canary fires, the vault rejects every ECDSA-authorized registry operation, and the binding can no longer change at all. □

S2.5 Legacy EOAs and Recovery Hard Forks

The binding gives users with funds still in legacy EOAs a pre-registered PQ identity, which is useful if Ethereum later adopts a recovery hard fork [4]. We emphasize that such funds are not protected by PQ-Vault itself (Proposition 2). The registry is not on the release path of escrowed funds, which deposits credit to β directly, and the canary is defense in depth: no theorem of the main paper relies on either.

S2.6 Transparent Proof Layer

If the transparent proof layer is enabled, a block carries a proof of correct state transition, which lets light validators check blocks without re-execution. Certificate soundness does not depend on that layer. The leader has no exclusive power over certification, since any party can aggregate signatures into a certificate (Algorithm 1).

S2.7 Trust in the Attestor Set: Further Discussion

With $W_{\mathsf{p}} = \lfloor W/3 \rfloor$, a coalition holding more than a third of the attestor stake could certify an arbitrary M_e without any honest signer; W_{p} trades certificate cost against the fraction of stake that must be corrupted. Comparing the cost of corrupting a third of the stake with the escrowed value is only an economic motivation for the assumption that corrupt stake stays below one third of every accepted attestor set. It would need further assumptions on utilities, bribery and enforceability, and an adversary may value disruption above the escrow. Signers of a fraudulent certificate are identifiable, but slashing them would require stake held on L1, which our prototype does not implement.

S3 Compact Certificates: Proofs and Details

S3.1 Proof of Theorem 5

The additive term accounts for the 128-bit modular reduction in Eq. (1): for any set $B \subseteq [W_{\mathsf{s}}]$, at most $|B|\lceil 2^{128}/W_{\mathsf{s}} \rceil$ of the 2^{128} values reduce into B. The verifier takes W and N from its own state and accepts only $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W$ (Algorithm 1), which bounds k_{rev} and makes the additive term negligible. The game fixes $W \le W_{\max} = 2^{64} - 1$ (stakes are 64-bit integers) and W_{p}, which the verifier takes from its state.

Theorem 5 (QROM soundness of compact certificates). *Let* $d = \lceil \log_2 N \rceil$*, let* $q_V = k_{\mathsf{rev}}(2d+3)$ *with* $k_{\mathsf{rev}} = k_{\mathsf{rev}}((1+\delta)W_{\mathsf{p}})$*, let* $t = q + q_V + 2^{d+1}$*, and let* h *be the smallest output width of the oracles and trees involved. For every adversary making* q *quantum queries to* ρ *and any number of signing queries,*

$$\mathsf{Adv}^{\mathsf{CERT\text{-}FORGE}}_{\mathcal{A}} \le 12t^2\varepsilon^* + \frac{48t^3 + 2q_V}{2^h} + 2\mathsf{Adv}^{\mathsf{uf}}_K.$$

The certificate is the BCS transformation [24] of a one-round public-coin IOP whose prover commits to the leaf vector and whose verifier draws the k_{rev} coins, and the proof follows the analysis of [34, Sec. 2.9]. Two features need care. The verifier's predicate involves signatures, which $\mathcal{A}$ obtains adaptively; and Eq. (1) derives each coin from a separate query.

Proof. Step 1 (one oracle value per commitment). Let $k_{\max} = k_{\mathsf{rev}}((1+\delta)W_{\mathsf{p}})$. Replace the oracle on coin inputs (DOM, j, x) with $j < k_{\max}$, where $x = (R_{\mathsf{sig}}, R_{\mathsf{att}}, W_{\mathsf{s}}, W_{\mathsf{p}}, N, M)$, by ρ_c with $hk_{\max}$-bit outputs: block j of $\rho_c(x)$ answers $\rho(\mathrm{DOM}\|j\|x)$, and the coin is its first 128 bits reduced modulo W_{s}. Distinct blocks are independent and uniform, so the oracle's distribution is unchanged. The domain-separated pair (ρ, ρ_c) is one oracle with inputs of two kinds, and Col refers to its h-bit Merkle outputs. A quantum query, possibly in superposition over both kinds of inputs and over j, is simulated by one query to the combined oracle: the output register is swapped into block j, the other blocks are set to $|+\rangle$, which XOR leaves unchanged, and the swap is undone. Query counts are therefore preserved. The verifier's queries are bounded by q_V, since $|P| \le k_{\mathsf{rev}}$ (Algorithm 1) and it computes at most k_{rev} coins and at most $2|P|(d+1)$ leaf and node hashes. Building R_{att} costs at most 2^{d+1} queries, and ML-DSA's internal hashing is independent of ρ (Section 3). Hence t bounds all queries to ρ and ρ_c in the experiment.

Step 2 (a fixed database property). Let D be a database without collisions. Consider each coin entry of D with input $x = (R, R_{\mathsf{att}}, W_{\mathsf{s}}, W_{\mathsf{p}}, N, M)$ whose W_{p} field is the game's W_{p} and whose W_{s} lies in $[(1+\delta)W_{\mathsf{p}}, W]$; the verifier reads

no other coin entries, and read the attestor leaves (pk_i, w_i) below R_{att} and the leaves (σ_i, L_i) below R from D, with $\perp$ for leaves not in D. Let V be the set of positions $i < N$ at which both leaves are present, $\mathsf{ML\text{-}DSA.Vf}(pk_i, M, \sigma_i) = 1$, $w_i > 0$ and $L_i + w_i \le W_{\mathsf{s}}$, and let $B = \bigcup_{i \in V} [L_i, L_i + w_i)$. Call the entry *light* if $w(V) \le W_{\mathsf{p}}$ and *hit* if all $k_{\mathsf{rev}}(W_{\mathsf{s}})$ coins lie in B. Let P be the property that D contains a light, hit coin entry. P is a fixed set of databases: it depends neither on the signing history nor on C, because signature validity is a fixed predicate once ML-DSA's independent oracle is fixed, and we condition on that oracle.

We bound $I(P \mid \overline{\mathsf{Col}}, t)$. Adding a coin entry for input x leaves every tree in D unchanged and creates one new entry. If that entry is light, then $|B| \le w(V) \le W_{\mathsf{p}}$, and each coin lies in B with probability at most $|B|\lceil 2^{128}/W_{\mathsf{s}} \rceil / 2^{128} \le W_{\mathsf{p}}/W_{\mathsf{s}} + W_{\mathsf{s}}/2^{128}$; the k_{rev} blocks are independent, so the entry is hit with probability at most $\varepsilon(W_{\mathsf{s}}) \le \varepsilon^*$. Adding a hash entry changes a tree below an existing root, and thereby V or B for some coin entry, only if its output equals one of the at most $2t$ child or root values occurring in inputs of D, which has probability less than $2t/2^h$. Hence $I(P \mid \overline{\mathsf{Col}}, t) \le \varepsilon^* + 2t/2^h$. With $I(\mathsf{Col}, t) < t/2^h$, Lemma 1(i) gives $\Pr[D \in P] \le 6t^2\varepsilon^* + 18t^3/2^h$ and $\Pr[D \in \mathsf{Col}] \le 6t^3/2^h$.

Step 3 (the database game). Suppose that the verifier accepts (M^*, cert^*) when every oracle value it reads is taken from D, that $D \notin \mathsf{Col}$, and that $w(C \cup S_{M^*}) \le W_{\mathsf{p}}$. The coin entry for cert^* is in D, and because D has no collisions the revealed leaves equal the leaves that D defines below R_{sig} and R_{att}. Every revealed position verifies and therefore lies in V, and every coin is covered, so the entry is hit. If it is light, $D \in P$. Otherwise $w(V) > W_{\mathsf{p}} \ge w(C \cup S_{M^*})$, so some $i \in V$ is an uncorrupted honest position that never signed M^*, while D holds a valid signature σ_i on M^* under its key. A reduction runs $\mathcal{A}$ with an efficient compressed-oracle simulation, forwards signing and corruption queries to the multi-user EUF-CMA challenger for the K honest keys, measures D at the end, and outputs (M^*, σ_i). Hence the database game is won with probability at most $\omega_D \le 6t^2\varepsilon^* + 24t^3/2^h + \mathsf{Adv}^{\mathsf{uf}}_K$.

Step 4 (back to the real oracle). The final check reads at most q_V oracle values, each of at least h bits. By Lemma 1(ii), $\mathsf{Adv} \le (\sqrt{\omega_D} + \sqrt{q_V/2^h})^2 \le 2\omega_D + 2q_V/2^h$, which is the claimed bound. □

S3.2 Remarks on Theorem 5

The proof needs no entropy property of honest signatures and no statement about *when* a leaf was committed: a valid leaf at an honest position is either charged to S_{M^*} or is a forgery extracted after $\mathcal{A}$ halts. Registered keys, including copies of honest keys, lie in C. Quantifying over all coin entries covers the adaptive choice of W_{s}, M^* and commitment. With $W_{\mathsf{p}} \ge W/4$, $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W < 2^{64}$ (stakes are 64-bit integers in the contract) and $\delta = 0.1$, we have $k_{\mathsf{rev}} < 2^{11}$ and $W_{\mathsf{s}}^2/(W_{\mathsf{p}}2^{128}) < 2^{-62}$, so the reduction term changes $\varepsilon(W_{\mathsf{s}})$ by a factor of at most $1 + 2^{-51}$.

S3.3 Proof of Theorem 6

Theorem 6 (Quantum grinding attack). *Let $d = \lceil \log_2 N \rceil$ and $c_{\mathsf{ev}} = d + 1 + k_{\mathsf{rev}}$. Suppose the adversary corrupts C with $a = w(C) \le W_{\mathsf{p}}$, receives no honest signatures, and some position outside C or some padding position exists. Fix any W_{s} with $(1+\delta)W_{\mathsf{p}} \le W_{\mathsf{s}} \le W$, let $\beta = |\{y < 2^{128} : y \bmod W_{\mathsf{s}} < a\}|/2^{128}$ and $\varepsilon_C = \beta^{k_{\mathsf{rev}}}$; then $\varepsilon_C \ge (a/W_{\mathsf{s}})^{k_{\mathsf{rev}}}$. If $q \ge 2^{d+1} + c_{\mathsf{ev}}$, there is a quantum adversary making at most q queries that wins* CERT-FORGE *with probability at least*

$$\frac{4}{\pi^2}(2T_G+1)^2\varepsilon_C\Big[1 - (2T_G+1)^2\big(\varepsilon_C + (d+2)2^{-h}\big)\Big],$$

where $T_G = \lfloor (q - 2^{d+1} - c_{\mathsf{ev}})/(2c_{\mathsf{ev}}) \rfloor$.

Proof. Write $2^{128} = mW_{\mathsf{s}} + r$ with $0 \le r < W_{\mathsf{s}}$. Exactly $am + \min(a, r) \ge a2^{128}/W_{\mathsf{s}}$ values reduce into $[0, a)$, which gives $\beta \ge a/W_{\mathsf{s}}$. The adversary signs the claimed M with every key in C and packs their intervals into $[0, a)$. The verifier checks only revealed intervals, so the claimed W_{s} need not equal the weight of the committed leaves. One position outside C carries an h-bit salt z, and all other leaves are fixed. Building the tree once costs at most 2^{d+1} queries. For each z, recomputing $R_{\mathsf{sig}}(z)$ along one root path and evaluating the k_{rev} coins costs c_{ev} queries. Let $G(z) = 1$ iff all coins are below a; a marked z yields a certificate that reveals only positions in C, all of which verify.

Let ν be the fraction of marked salts. Fix the oracle values on Merkle inputs, which determine $R_{\mathsf{sig}}(\cdot)$. Coin inputs are domain-separated from Merkle inputs, so the indicators for distinct roots are independent with mean ε_C, and salts with equal roots share their indicator. Two distinct salts give equal roots with probability at most $(d+1)2^{-h}$, because the first equal hash on their paths is a collision. Hence $\mathbb{E}[\nu] = \varepsilon_C$ and $\mathbb{E}[\nu^2] \le \varepsilon_C^2 + \varepsilon_C(d+2)2^{-h}$. Grover's algorithm with T_G iterations uses $2T_Gc_{\mathsf{ev}}$ queries (computation and uncomputation) and c_{ev} more to recompute the certificate classically. For a fixed oracle, it outputs a marked salt with probability $\sin^2((2T_G+1)\theta)$, where $\sin^2\theta = \nu$. If $\nu \le \nu_{\max} = (2T_G+1)^{-2}$, then $(2T_G+1)\theta \le \pi/2$ (since $\arcsin x \le \pi x/2$), and $\sin y \ge 2y/\pi$ and $\theta \ge \sqrt{\nu}$ give success at least $(4/\pi^2)(2T_G+1)^2\nu$. Averaging over the oracle, the success probability is at least $(4/\pi^2)(2T_G+1)^2\big(\mathbb{E}[\nu] - \mathbb{E}[\nu\,\mathbf{1}_{\nu>\nu_{\max}}]\big)$, and $\mathbb{E}[\nu\,\mathbf{1}_{\nu>\nu_{\max}}] \le \mathbb{E}[\nu^2]/\nu_{\max}$ gives the bound. □

For the grinding term, the best attack's success probability in the example of Corollary 7 lies between $2^{-81.1}$ and $2^{-61.5}$; only the factor $3\pi^2c_{\mathsf{ev}}^2$ remains open. The attack is a query-model statement.

S3.4 Deployed Parameters: Algorand State Proofs

Algorand state proofs [15], to our knowledge the only deployed compact certificates, use consensus parameters [47] with a strength target of 256, documented as "$k + q$ (for pre-quantum security) or $k + 2q$ (for post-quantum security)", and a proven weight of 30% of the top 1024 accounts. Coins come from one SHAKE256 stream with rejection sampling, without the reduction term of Eq. (4). Read as $\lambda + 2\log_2 q$ with $\log_2 q = 64$, this target matches our rule up to the constant κ_Q. If its hash functions are modeled as random oracles (an assumption for its SumHash-based commitments), the grinding term of Theorem 5 certifies about $256 - 128 - \log_2 12 \approx 124$ bits against quantum provers making 2^{64} queries. We found no vulnerability: our theorem justifies its heuristic $k + 2q$ choice for the grinding term, with a gap of 3.6 bits. Systems that follow the classical $k + q$ rule of [14], as the prototype of [38] did, fall under Corollary 7.

S3.5 The Exact Parameter Rule

The rule (5) controls the *grinding* term only. For an arbitrary total query budget T and a per-epoch grinding budget η_g, the exact sufficient rule is

$$k_{\mathsf{rev}}(W_{\mathsf{s}}) \geq \left\lceil \frac{\log_2(12T^2/\eta_g)}{-\log_2(W_{\mathsf{p}}/W_{\mathsf{s}} + W_{\mathsf{s}}/2^{128})} \right\rceil,$$

which Eq. (5) implies for $\eta_g = 2^{-\lambda}$ in the regime $T \leq q(1+2^{-20})$ and for $W_{\mathsf{p}} \geq W/4$, as deployed, because $12(1+2^{-20})^2(1+2^{-51}) < 16$ (Section S3.2). T counts all queries to the hash oracle on the L1 path (256- or 384-bit): coins, certificate trees, C_{cert}, checkpoint headers, withdrawal trees and code hashes. Scores and order keys use domain-separated inputs, which we model as independent oracles.

S3.6 Output Width and the Commitment Term

The other terms of Theorem 5 need separate budgets. In Theorem 5, h is the *smallest* output width of the oracles and trees involved, because Col and the attach events range over all of them; widening only R_{sig} does not change h while R_{att} stays at 256 bits. The commitment term has the form of the generic collision bound [41]. It is an upper bound, not an attack. Generic quantum collision search is no cheaper than classical 2^{128} search in realistic cost models [40], [48], but that does not make the query-model term 2^{-128}. Table 3 combines all terms of Theorem 20 for two instantiations. The deployed vault uses Keccak-256 for every tree and for C_{cert}. A uniformly widened variant uses 384-bit nodes everywhere (Keccak-384 off chain, two domain-separated Keccak-256 outputs per node on L1; not implemented). Our `pqcert` implementation supports Keccak-384 certificate trees, which enlarge Dilithium2 (r3) certificates by 4–7% at unchanged verification time at $N = 2^{12}$–2^{16} (Table 6).

S4 Leader Election: Proofs and Details

This section contains the full proofs of the results of Section 7 and material moved from that section.

S4.1 Exponential Race and Steering

Lemma 8 (Exponential race). *Let $U_1, \ldots, U_n$ be independent and uniform on $(0,1]$, and let $s_i = -\log_2 U_i / w_i$. Then the s_i are independent with $s_i \sim \mathrm{Exp}(w_i \ln 2)$. Consequently, (i) $\Pr[i \text{ has the lowest score}] = w_i/W$, and (ii) splitting stake w into identities w', w'' with $w' + w'' = w$ leaves the coalition's winning probability unchanged (Sybil invariance). With h-bit values $u_i = \lceil U_i 2^h \rceil / 2^h$ and ties broken by the smaller identity, the winner differs from the continuous winner with probability at most $N(N-1)2^{-h}$.*

Proof. If U is uniform on $(0,1]$, then $-\ln U \sim \mathrm{Exp}(1)$ and hence $-\log_2 U/w_i \sim \mathrm{Exp}(w_i \ln 2)$. For independent exponentials with rates r_i, the minimum is attained at i with probability $r_i / \sum_l r_l$, which gives (i); the minimum of $\mathrm{Exp}(r')$ and $\mathrm{Exp}(r'')$ is $\mathrm{Exp}(r' + r'')$, which gives (ii). For the discretization, if the continuous winner i loses to j after rounding, then $U_j < U_i^{w_j/w_i} \leq u_i^{w_j/w_i} \leq u_j < U_j + 2^{-h}$. So U_j lies in an interval of length 2^{-h} below $U_i^{w_j/w_i}$, which has probability at most 2^{-h} for each of the $N(N-1)$ ordered pairs. □

Lemma 8 is the weighted-sampling identity of [42] applied to the score of [7]. It is a statement about *unselected* uniforms. If a party picks one of several inputs after seeing the oracle's outputs, the selected output is not uniform: keeping the larger of two independent uniforms gives a value with CDF u^2, although both inputs were distinct.

Proposition 9 (m-way steering). *Suppose that, before slot T, the adversary can choose $\mathfrak{S}_T$ among m candidates; that under each candidate its own best score is a fresh $\mathrm{Exp}(\alpha)$ variable (rates in units of $\ln 2$), independent across candidates; and that the honest best score is $\mathrm{Exp}(1-\alpha)$ and unobserved when it chooses. Picking the candidate that minimizes its own best score is optimal and leads slot T with probability $p_m(\alpha) = m\alpha/(1+(m-1)\alpha)$.*

Proof. For a fixed candidate with adversarial best score $X = x$, the adversary wins with probability $p = e^{-(1-\alpha)x}$, which decreases in x; minimizing x is therefore optimal given its information. Writing $b = \alpha/(1-\alpha)$, we get $\Pr[p \leq v] = \Pr[X \geq -\ln v/(1-\alpha)] = v^b$, that is, $p \sim \mathrm{Beta}(b,1)$. The best of m independent candidates yields $\max_l p_l$, whose CDF is v^{mb} and whose mean is $mb/(mb+1) = p_m(\alpha)$. □

Corollary 10 (Steering under the runner-up rule). *Consider proposer-dependent seeds in the race without threshold of [7], and a single-identity adversary that uses only its decision at slot $T-1$. (a) If it can observe the honest runner-up's reveal before its publish-or-skip decision becomes irrevocable, for example as a rushing adversary when honest proposals are broadcast at the start of the slot, it captures slot T with probability $\alpha + \alpha^2(1-\alpha)/(1+\alpha)$. (b) If that reveal becomes known only afterwards, it captures slot T with probability $\alpha\big(\alpha + (1-\alpha)\alpha^{1/(1-\alpha)}\big) + (1-\alpha)\alpha$. Both exceed α. Splitting into more identities raises the probability that the adversary holds the top $L \geq 2$ ranks, and hence its option count; Sybil invariance therefore fails.*

Proof. The adversary leads slot $T-1$ with probability α (Lemma 8); otherwise the seed of slot T is fresh and its capture probability is α. In case (a), it knows both candidate seeds, its own and the runner-up's, and Proposition 9 with $m = 2$ gives $p_2(\alpha) = 2\alpha/(1+\alpha)$; the total is $\alpha p_2(\alpha) + (1-\alpha)\alpha$. In case (b), skipping yields an unknown fresh seed with capture probability α, while publishing yields probability $p \sim \mathrm{Beta}(b,1)$ as in Proposition 9, known to the adversary; the best choice achieves $\mathbb{E}[\max(p,\alpha)] = \alpha + \int_\alpha^1 (1-v^b)\,dv = \alpha + (1-\alpha)\alpha^{1/(1-\alpha)}$. Since $\mathbb{E}[\max(p,\alpha)] > \mathbb{E}[p] = \alpha$, both totals exceed α. If the adversary holds the top L ranks of slot $T-1$, it can choose among up to $L+1$ seeds, and $p_{L+1}(\alpha)$ increases with L. Splitting a fixed stake leaves its probability of holding rank 1 unchanged (Lemma 8) but raises its probability of holding ranks 1 and 2 together. □

S4.2 Ideal Model

Theorem 11 (Leader election in the ideal model). *In any m-ideal experiment with continuous scores, the following hold. (a) Given the reveal set, the proposer ranking of each slot is a fixed public function of the table. No adversarial action within the epoch (skipping, withholding proposals, choosing among its identities) changes any ranking, except by removing the adversary's own identities. If $m = 1$, slot leaders are independent across slots and distributed as in Lemma 8. (b) The adversary chooses the table among at most $m \leq 2^c$ candidates by withholding a subset of*

its late reveals. A slot is not guaranteed with probability at most p_B under each candidate, and the adversary can propose only in such slots. It proposes in a fixed target slot with probability at most $1-(1-p_B)^m$, and its expected fraction of the k slots of the epoch is at most $p_B+\sqrt{c\ln 2/(2k)}$. (c) The probability that none of n specified slots is guaranteed is at most $\min\{1, mp_B^n\}$. For a window of n consecutive slots whose start the adversary may choose after seeing the schedule, it is at most $\min\{1,(k-n+1)mp_B^n\}$.

Proof. For part (a), the ranking of slot j sorts row j of the table, which no action within the epoch changes. Removing its own identities cannot help the adversary. For $m=1$, all scores are independent and uniform, and Lemma 8 gives the leader distribution and independence across slots.

We next bound one slot under one candidate table. Given the first $r-1$ ranks, rank r is adversarial with probability $(\alpha-a)/(1-a-g)$, where a and g are the adversarial and honest stake already ranked. This is at most $\alpha/(1-g)$, since $(\alpha-a)(1-g)\le\alpha(1-a-g)$ is equivalent to $a(\alpha+g-1)\le 0$. For $r\le B+1$, at most B honest identities have been ranked, so $g\le Bw_{\max}$ and the probability is at most α_B. Hence the first $B+1$ ranks are all honest with probability at least $(1-\alpha_B)^{B+1}$, and a slot is not guaranteed with probability at most p_B. In a guaranteed slot, at most B of the first $B+1\le L+1$ ranks are suppressed, and the best-ranked surviving one is honest and proposes.

For part (b), these per-candidate bounds hold independently across candidates, so the adversary proposes in the target slot under at least one candidate with probability at most $1-(1-p_B)^m$; no selection rule does better. For the share, let $\bar{X}_l=\frac{1}{k}\sum_j X_{l,j}$, where $X_{l,j}$ indicates that slot j is not guaranteed under candidate l. Under each candidate, the $X_{l,j}$ are independent with mean at most p_B. By Hoeffding's lemma, each $\bar{X}_l$ minus its mean is sub-Gaussian with variance proxy $1/(4k)$. The maximal inequality for sub-Gaussian variables, which requires no independence between candidates, gives $\mathbb{E}[\max_{l\le m}\bar{X}_l]\le p_B+\sqrt{\ln m/(2k)}$, and $\ln m\le c\ln 2$.

For part (c), the event that none of the n slots is guaranteed under the selected table is contained in the union of the same events under the m candidates. Under each candidate, slots are independent, and each fails with probability at most p_B. For an adversarially placed window, we take a further union bound over the $k-n+1$ possible starts. □

Lemma 12 (Reveals are included). *Suppose that an honest proposer includes every reveal it received $\Delta_{\mathcal{S}}$ before its slot, that messages are delivered within $\Delta_{\mathcal{S}}$ and a slot lasts at least $(L+2)\Delta_{\mathcal{S}}$, and that the finality gadget finalizes each block of the reveal window before the next slot starts. If a phase or sub-window of ℓ slots has a guaranteed slot after its first slot, every honest reveal of that phase or sub-window is included. Under each candidate table of the epoch that contains the window, this fails with probability at most $p_B^{\ell-1}$.*

Proof. An honest validator sends its reveal when its phase or sub-window starts, so every honest proposer of the second and later slots of the window has received it in time, and the proposer of a guaranteed slot is honest. The positions of the windows are fixed by the order, and under each candidate table the slots are independent, each not guaranteed with probability at most p_B by Theorem 11. □

Proposition 13 (Late reveals are rare). *Consider consecutive m-ideal epochs, each with respect to its own c, starting from an epoch with $c=0$. Let $\mu=(1-2\alpha)/(1-3\alpha)$, and let OK_e be the event that* Fail *occurs in none of the epochs up to e. Then, for every epoch e and every $x\ge 1$,*

$$\Pr[c_e\ge x\wedge\mathsf{OK}_e]\le\mu\alpha^x,\qquad \mathbb{E}[2^{c_e}\mathbf{1}_{\mathsf{OK}_e}]\le\mu,$$
$$\Pr[\mathsf{OK}_{e-1}\wedge\neg\mathsf{OK}_e]\le\delta_{\mathsf{LE}},$$

where $\delta_{\mathsf{LE}}=\mu(\alpha^{L_t}+L_t p_B^{\ell_s-1}+p_B^{\ell_0-1})$. The adversary proposes in a fixed target slot of epoch e with probability at most $\pi_{\mathsf{cap}}+e\delta_{\mathsf{LE}}$, where $\pi_{\mathsf{cap}}=p_B+\sum_{x\ge1}\min\{1,\mu\alpha^x\}\big((1-p_B)^{2^{x-1}}-(1-p_B)^{2^x}\big)$.

Proof. Write $m_e=2^{c_e}$. The registrations for epoch e and, by mild adaptivity, the corrupt set acting during its reveal window, which lies in epoch $e-1$ and was therefore requested by epoch $e-4$, are fixed before the candidate tables of epoch $e-1$ are sampled at that epoch's time τ^*, in epoch $e-2$. Under each candidate table of epoch $e-1$, the order of epoch e is a stake-weighted sample without replacement. As in the proof of Theorem 11 with $g=0$, its first x ranks are all adversarial with probability at most α^x. In particular, the tail contains no honest identity with probability at most α^{L_t}, and by Lemma 12 some of the L_t sub-windows, which lie in epoch $e-1$ at positions fixed by the order, has no guaranteed slot after its first slot with probability at most $L_t p_B^{\ell_s-1}$; for the common phase, the bound is $p_B^{\ell_0-1}$. On OK_{e-1}, the adversary selects one of m_{e-1} candidates, and the candidate tables are independent of m_{e-1}. Summing over candidates gives $\Pr[c_e\ge x\wedge\mathsf{OK}_e]\le\mathbb{E}[m_{e-1}\mathbf{1}_{\mathsf{OK}_{e-1}}]\alpha^x$ and $\Pr[\mathsf{OK}_{e-1}\wedge\neg\mathsf{OK}_e]\le\mathbb{E}[m_{e-1}\mathbf{1}_{\mathsf{OK}_{e-1}}](\alpha^{L_t}+L_t p_B^{\ell_s-1}+p_B^{\ell_0-1})$. On OK_e, no honest reveal is missing, so suspensions remove only adversarial identities and do not raise α. Since $2^c=1+\sum_{x=1}^{c}2^{x-1}$,

$$\begin{aligned}\mathbb{E}[2^{c_e}\mathbf{1}_{\mathsf{OK}_e}]&\le 1+\sum_{x\ge1}2^{x-1}\Pr[c_e\ge x\wedge\mathsf{OK}_e]\\&\le 1+\mathbb{E}[m_{e-1}\mathbf{1}_{\mathsf{OK}_{e-1}}]\frac{\alpha}{1-2\alpha},\end{aligned}$$

and induction from $m=1$ gives the bound μ, the fixed point of $y\mapsto 1+y\alpha/(1-2\alpha)$. The capture bound averages Theorem 11(b) over c_e on OK_e, and adds $\Pr[\neg\mathsf{OK}_e]\le e\delta_{\mathsf{LE}}$. □

S4.3 From the Ideal Model to the Protocol

Lemma 14 (Reprogramming hidden inputs). *Let a quantum process make q queries to an oracle O, and let its run with O be determined by randomness independent of a random variable Z, so that, with that randomness fixed, its state before each query does not depend on Z. Let S_Z be a set of inputs, determined by Z, with $\Pr_Z[x\in S_Z]\le p$ for every input x, and let O' agree with O outside S_Z. Then the final states of the process with O and with O' have expected trace distance at most $2q\sqrt{p}$.*

Proof. Purify the process, including all parties and their classical interaction, and defer all measurements; queries made in parallel are counted separately. Fix all randomness except Z. Let $|\phi_i\rangle$ be the state before the i-th query when the oracle is O, and let Π project the query register onto S_Z. Replacing the oracle by O' one query at a time, and

using that later operations preserve norms, gives $\||\psi^O\rangle - |\psi^{O'}\rangle\| \le \sum_{i=1}^{q} \|(U_O - U_{O'})|\phi_i\rangle\| \le 2\sum_i \|\Pi|\phi_i\rangle\|$, because U_O and $U_{O'}$ act identically on inputs outside S_Z. Since $|\phi_i\rangle$ does not depend on Z, $\mathbb{E}_Z\|\Pi|\phi_i\rangle\| \le (\mathbb{E}_Z\|\Pi|\phi_i\rangle\|^2)^{1/2} = (\sum_x \|\Pi_x|\phi_i\rangle\|^2 \Pr[x \in S_Z])^{1/2} \le \sqrt{p}$. The trace distance of pure states is at most their Euclidean distance. □

Theorem 15 (Ideal to real, QROM). *Model the commitment and seed functions of QPoS-S, cSHAKE256 with distinct tags, as a quantum random oracle independent of the score oracle ρ and of ML-DSA's internal hashing. Let commitments, secrets and seeds have n_c, n_r and n_s bits, and let identities and epochs be fixed-length fields of n_p bits in total. Assume mildly adaptive corruption (Section 4), the conditions of Lemma 12, that* Fail *does not occur, and that $c \le \bar{c}$; on $\neg$Fail, $\bar{c} = L_t - 1$ suffices. Let $m = 2^{\bar{c}}$. Let t count all oracle queries made by all parties before τ^*. Then there is an m-ideal experiment such that, for every $[0,1]$-valued function F of the epoch's leaders and the adversary's final output,*

$$\big|\mathbb{E}_{\mathsf{real}}[F] - \mathbb{E}_{\mathsf{ideal}}[F]\big| \le \epsilon_{\mathsf{lead}} = \frac{2^{n_p+2n_r}}{2^{n_c+1}} + \frac{2t}{2^{n_r/2}} + \frac{2t\sqrt{m}}{2^{n_s/2}} + \frac{m^2}{2^{n_s+1}}.$$

Queries made after τ^* do not appear in the bound.

Proof. Write ρ_c for the commitment and seed oracle. Index the possible reveal sets by bitmasks $b \in \{0,1\}^{\bar{c}}$ over the late adversarial identities in reveal order, and let X_b denote the seed input of reveal set b.

Step 0 (binding). Modify the protocol so that a reveal is accepted only if it is the lexicographically smallest opening of its commitment. The modified and real experiments are identical unless, for some identity-and-epoch prefix p, the map $r \mapsto \rho_c(\text{COMMIT} \,\|\, p \,\|\, r)$ is not injective on $\{0,1\}^{n_r}$. This event depends only on the uniform oracle and has probability at most $2^{n_p} \cdot 2^{2n_r}/2^{n_c+1}$. In the modified protocol, each adversarial identity can only reveal its unique admissible opening or withhold it. The admissibility check for an identity $ID_j \neq ID_{i^*}$ reads ρ_c only on commitment inputs with that prefix, which lie outside the set S of Step 1; in Steps 1 and 2 these values are part of the fixed randomness. The reveal list, and hence each X_b, is thus a function of b, of the commitments and honest secrets, and of ρ_c on commitment inputs. Registrations, which also fix the stakes, close before the reveal window opens, so all commitments and weights are fixed before τ^*. Every reveal before the sub-window of i^* is final at τ^*, so the adversary's only remaining choice is which late identities reveal. The reveal order is a function of the previous seed, which is public before τ^*. By mild adaptivity, the corrupt set during the reveal window was requested before that seed was known, so i^* is honest until after τ^* and is part of the fixed randomness as well.

Step 1 (hiding the last honest secret). Let $r = r_{i^*}$. Let S be the set of commitment inputs with fields (ID_{i^*}, e, r), together with the seed inputs whose entry for ID_{i^*} equals r. Fixed-length fields and the strictly ordered encoding of the reveal list give $\Pr_r[x \in S] \le 2^{-n_r}$ for every input x. Sample a function H on commitment and seed inputs, an independent function W on seed inputs, and a uniform value a. Let G map the commitment input of i^* to a, agree with W on the seed inputs in S, and agree with H elsewhere. For every fixed r, G is a uniform oracle, so the experiment of Step 0, in which $\rho_c = G$ throughout and $a_{i^*} = a$, is distributed as before.

Let Hyb_A answer commitment and seed queries before τ^* with H and later queries with G. Validator i^* uses r before τ^* only through $a_{i^*} = a$, so we keep r outside the process. Given (H, W, a) and the other randomness, the run before τ^* in Hyb_A therefore does not depend on r. Lemma 14 with $Z = r$ bounds the trace distance of the states at τ^* by $2t\,2^{-n_r/2}$. The execution after τ^* is the same map in both experiments, so the bound carries over to the final states, however many queries follow.

Step 2 (fresh scores). Call b *realizable* if its bits name existing late identities that all have an admissible opening. In Hyb_A, the points X_b of realizable b are distinct seed inputs in S; for other b, the value Z_b below is an unused dummy. They are determined by r and by the run before τ^*, which uses H and a but not W. Their values $W(X_b)$ are therefore independent uniforms, independent of (H, a, r) and of that run. Hence Hyb_A is unchanged in distribution if we sample independent uniform Z_b in advance and set $W(X_b) = Z_b$, so that the seed of reveal set b is $\sigma_b = Z_b$. Let S' be the set of score and next-epoch order inputs whose seed field equals some Z_b. S' is a function of $Z = (Z_b)_b$ alone, and $\Pr_Z[x \in S'] \le m2^{-n_s}$ for every input x. Let Hyb_B be Hyb_A with fresh uniform values V' on S' for queries after τ^*. Because S' depends only on Z, which is independent of the score oracle ρ, replacing the values of ρ on S' by V' both before and after τ^* leaves Hyb_A unchanged in distribution. The result differs from Hyb_B only in the answers on S' before τ^*. The run of Hyb_B before τ^* does not depend on Z, so Lemma 14 bounds the distance by $2t\sqrt{m}\,2^{-n_s/2}$.

Step 3 (the ideal experiment). In Hyb_B, the seeds Z_b are distinct except with probability $m^2/2^{n_s+1}$. The score table and next-epoch order keys of reveal set b are then V' on the inputs of S' with seed Z_b, a family of independent uniform tables, independent of everything before τ^* and accessible to the adversary afterwards. Hyb_B is therefore m-ideal, and the triangle inequality gives the bound, because trace distance bounds the difference of expectations of $[0,1]$-valued functions. □

Corollary 16 (Real protocol). *Let $m = 2^{L_t-1}$ and $\epsilon_{\mathsf{LE}} = \epsilon_{\mathsf{lead}} + \epsilon_{\mathsf{disc}}$, where $\epsilon_{\mathsf{disc}} = (k+1)mN(N-1)2^{-h}$ applies Lemma 8 to the k slot races and the order race of each candidate, and N counts all registered identities. Against quantum adversaries with mildly adaptive corruption, under the conditions of Lemma 12, every bound of Theorem 11, Proposition 13, and Lemma 12 holds in the real protocol, for the first E epochs, up to an additive $2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$. In particular, the adversary proposes in a fixed target slot with probability at most $\pi_{\mathsf{cap}} + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$, and an epoch has no guaranteed slot among n consecutive slots placed by the adversary with probability at most $(k-n+1)\mu p_B^n + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$.*

Proof. Replace the epochs by m-ideal ones one at a time. Each step applies Theorem 15, whose function F may depend on the entire later run, and Lemma 8. Its premises fail only on $\neg\mathsf{OK}_e$, which has probability at most δ_{LE} given OK_{e-1} by Proposition 13, applied to the already idealized earlier epochs. Each step therefore costs at most $\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}}$, and $\Pr[\neg\mathsf{OK}_E] \le E\delta_{\mathsf{LE}}$ adds the rest. The bounds then hold for consecutive m-ideal epochs, and the window bound averages Theorem 11(c) over m_e with $\mathbb{E}[m_e \mathbf{1}_{\mathsf{OK}_e}] \le \mu$. □

S4.4 Additional Remarks

S4.4.0.1 Proposer-dependent seeds: In the hash-chain protocol of [7] and in the sidechain design of [18], each validator's score is computed from its own secret hash-chain value, so scores are private until their owners propose, and the seed is updated as $\mathfrak{S}_{j+1} = \mathsf{H}(\mathfrak{S}_j \,\|$ reveal of the *actual* proposer of $j)$, and a skipped leader is replaced by the runner-up. A coalition can then influence $\mathfrak{S}_{j+1}$. If it leads slot j, it can publish its own block or skip and let the runner-up publish. If it holds the top L positions, it can make any of $L+1$ proposers publish. Because the adversary knows its own future chain values, it can compute its *own* best score in the next slot for each candidate seed. Honest scores remain unpredictable.

S4.4.0.2 Target slots: Target slots matter because roles are attached to slot indices, for example the N_{cp}-th block that fixes a checkpoint, or the slot whose block includes or censors a dispute-relevant transaction.

S4.4.0.3 Fallback proposers: The fallback proposers matter only under denial of service. Without them ($L = 0$), suppressing the leader of every slot leaves the sidechain without honest blocks; with $L \geq B$, a slot loses its honest proposer only if it is not guaranteed. A non-guaranteed slot may still have an honest proposer, but an adversarial proposer of better rank can take the slot and censor it.

S4.4.0.4 Reveal order: The order of reveals keeps c small. Because rank 1 of the order is drawn in proportion to stake, Sybil splitting does not help the adversary obtain the last positions, and the adversary can influence the order only through its choice among the previous epoch's candidates.

S4.4.0.5 Parameters of Theorem 15: QPoS-S uses $n_r = n_s = 512$, $n_c = 1536$ and 64-bit identity and epoch fields ($n_p = 128$). For $t = 2^{64}$ and $\bar{c} = L_t - 1 = 24$, the four terms are 2^{-385}, 2^{-191}, 2^{-179} and 2^{-465}, so $\epsilon_{\mathsf{lead}} \leq 2^{-178.9}$. The value $\bar{c} = 24$ is a worst case; by Proposition 13, c is usually 0 or 1. Scores and order keys use H with $h = 256$. The wide secrets and seeds keep the two hiding terms small. The wide commitments make binding statistical in the QROM, so the proof needs no argument about computationally binding commitments against quantum adversaries. In practice, SHAKE-based binding is capped at the function's 256-bit security claim.

S4.5 Consequences in Detail

Part (c) does not multiply per-slot probabilities, which is invalid after seed selection. Since the schedule is public, the adversary can time an action, such as the rebinding of Proposition 4, to precede a run of non-guaranteed slots. By Corollary 16, an honest transaction is still included within n slots of any start chosen after the schedule is public, except with probability $\zeta + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$, whenever $n \geq \ln(k\mu/\zeta)/(-\ln p_B)$. This high-probability guarantee is the sidechain liveness that Proposition 4 uses. Turning inclusion into finality additionally needs the liveness of the chosen finality gadget, and a window that straddles an epoch boundary adds the bounds of both epochs. For $\alpha = 0.3$ and $k = 2000$, $\zeta = 2^{-40}$ needs $n = 31$ slots without denial of service and $n = 56$ with $B = 1$. For $E = 2^{20}$ epochs, $2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}}) < 2^{-19}$ at $\alpha = 0.3$ and $< 2^{-36}$ at $\alpha = 0.2$, with or without $B = 1$. Larger L_t or ℓ_s lowers δ_{LE} further at the cost of a longer reveal window. The window of $1{,}352$ slots occupies about two thirds of a $2{,}000$-slot epoch; reveals are small transactions carried by ordinary blocks, so this costs block space, not slots. A validator whose reveal is missing is suspended, but the adversary withholds only when the honest seed does not already suit it, so suspension is not a deterrent in our analysis. PQ-Vault attaches no exclusive role to any single slot: certificates can be aggregated by anyone, and checkpoints are certified by attestors rather than by a leader.

S4.6 Long-Range Attacks

A posterior corruption of retired keys allows the adversary to build an alternative history [31], [49]. Signature unforgeability, on which the analysis of [18] relied, does not prevent this. In PQ-Vault, Vault accepts only checkpoints for the epoch after its latest finalized checkpoint, verified against the attestor commitment R^e_{att} fixed by that checkpoint. Bootstrapping nodes use the vault log as their weak-subjectivity source. Under Assumption A1, an alternative history that forks before the latest anchored checkpoint is rejected outright. A fork after it requires a certificate for the current attestor set, whose members are still bonded by our unbonding condition; forging one is bounded by Theorems 5 and 19.

S5 L1 Acceptance: Dispute Protocol and Proofs

This section supplements Section 8. It gives the full specification of the optimistic dispute protocol, the detailed timing discussion, and the full proofs of Proposition 17, Lemma 18, and Theorems 19 and 20.

S5.1 Record Commitment

The relayer commits to the records in a Merkle tree C_{cert} of depth $\lceil \log_2 k_{\mathsf{rev}} \rceil$, with r_j at index j. Its leaf hashes a fixed-length encoding of r_j that includes the hashes of σ_{i_j} and of both paths.

S5.2 State Machine

A proposal is created by propose (Section 5.1) at time τ_0 and is *pending* until it is finalized, rejected or stale. The relayer publishes the records as blob data. Safety never relies on blobs: a watcher that cannot fetch the records or match them to C_{cert} uses the open-all challenge. Its *challenge deadline* is $\tau_0 + T_{\mathsf{ch}}$, extended to $\max\{\tau_0 + T_{\mathsf{ch}}, \tau_{\mathsf{post}} + \Delta_{\mathsf{post}}\}$ if an open-all posting completed at τ_{post}. Times are block timestamps at inclusion. A challenge or open-all request is accepted only at timestamps strictly before its deadline, and a response or posting at timestamps up to and including its deadline; finalize is accepted from the challenge deadline on, and timeout and timeoutAll only strictly after their deadlines. Hence no timestamp admits both an action and the transition that its deadline unlocks. The transitions are as follows. (a) challenge(j), with bond b_C, before the challenge deadline and at most once per index, opens a coin challenge with response deadline $\tau + \Delta_{\mathsf{resp}}$. (b) respond$(j, r, addr)$ by any party up to and including that deadline first requires that r authenticate under C_{cert} at index j, and otherwise *reverts*.

If the committed data fail (i) or (ii), the proposal is *rejected*. Next, the call reverts unless the code hash of $addr$ equals η_{i_j}. Finally, a failing signature rejects the proposal, and a valid one closes the challenge. (c) timeout(j) after an unanswered response deadline rejects the proposal. (d) challengeAll, at most once per proposal and before the challenge deadline, requires all records to be posted as calldata within Δ_{open}. postRecords accepts records that authenticate under C_{cert} without evaluating Local, and sets τ_{post} once all k_{rev} are posted. If posting is incomplete at its deadline, timeoutAll rejects the proposal. (e) finalize requires that the challenge deadline has been reached, that no coin challenge is open, and that no posting is pending. The proposal is then finalized if its epoch is $e^* + 1$, and stale otherwise. A proposal is thus rejected only on an authenticated record that fails Local or on an expired deadline; a malformed record, wrong path or wrong key address from a third party only reverts. On rejection, the challenger is credited b_R and its bond; a correct answer credits b_C to the relayer. Payments are pulled by their recipients, so a recipient that refuses transfers cannot block a transition. An open-all request carries no bond, so a withholding relayer cannot charge honest watchers; in exchange, anyone can force each proposal to post its records once, at a cost to the relayer of 3.0×10^7 gas (Section 9.5). Unavailable data cannot be attributed to either party, so one of them must bear this cost; we protect watchers.

S5.3 Timing

Let Δ_{chk} bound the time an honest watcher needs to observe a proposal or a completed posting, fetch the records and evaluate Local on all of them, and let Δ_{prep} bound the relayer's time to prepare a response or posting. All Δ's bound differences of inclusion timestamps. We require the strict inequalities

$$T_{\mathsf{ch}} > \Delta_{\mathsf{chk}} + \Delta_{\mathsf{inc}}, \qquad \Delta_{\mathsf{post}} > \Delta_{\mathsf{chk}} + \Delta_{\mathsf{inc}},$$

which match the strict guards on challenges, and $\Delta_{\mathsf{open}}, \Delta_{\mathsf{resp}} \geq \Delta_{\mathsf{prep}} + \Delta_{\mathsf{inc}}$, which match the inclusive guards on postings and responses; the relayer submits all posting transactions (seven in our measurements) concurrently. Every pending proposal is then *eligible* for a final transition (finalize, timeout or timeoutAll) by $T_{\mathsf{fin}} = T_{\mathsf{ch}} + \Delta_{\mathsf{open}} + \Delta_{\mathsf{post}} + \Delta_{\mathsf{resp}}$ after its inclusion. A transition executes only when some party submits it. If an honest party submits it once it is eligible, it is included by $T_{\mathsf{fin}} + \Delta_{\mathsf{inc}}$. Safety never depends on these calls, since a proposal with an open challenge or pending posting cannot be finalized. A stale proposal has no effect on the vault's state from the moment another proposal for its epoch is finalized; only its bond refund needs a call. The worst case is an open-all request just before $\tau_0 + T_{\mathsf{ch}}$, followed by a coin challenge just before the extended deadline. Our contract uses $T_{\mathsf{ch}} = 7$ days, $\Delta_{\mathsf{open}} = \Delta_{\mathsf{resp}} = 1$ day and $\Delta_{\mathsf{post}} = 2$ days, in line with optimistic rollups [26], so $T_{\mathsf{fin}} \leq 11$ days. Each coin index can be challenged at most once, so at most k_{rev} responses are ever needed per proposal.

S5.4 Proofs

Proposition 17. *If* Vault *finalizes a checkpoint whenever a Groth16 proof* Π *of "all certificates verify" is valid, then a QPT adversary can finalize an arbitrary checkpoint, and with it arbitrary withdrawals, with probability* $1 - \mathsf{negl}$.

Proof. The common reference string contains $[\tau^i]_1$ and $[\tau]_2$ for a trapdoor τ. Shor's algorithm computes τ from $[\tau]_1$. With τ (and the other trapdoor elements, computed the same way), the Groth16 simulator outputs a valid proof for any statement, true or false. The vault accepts it. □

Lemma 18 (Local decomposition). *Except for a collision of* H, $\mathsf{Local}(j, r_j) = 1$ *for all* $j < k_{\mathsf{rev}}$ *if and only if the distinct positions* $P = \{i_j\}$*, their leaves, and the multiproofs assembled from the single paths form a certificate that Algorithm 1 accepts, instantiated with the vault's leaf encodings.*

Proof. If two records name the same position, both of their leaves open there under R_{sig} and R_{att}, so they are equal unless the paths contain a collision. Hence P is well defined with $|P| \leq k_{\mathsf{rev}}$, every position covers its coin, and the single paths contain every sibling that the multiproofs need. The interval, coverage, signature and opening checks of Algorithm 1 are exactly (i)–(iii) taken over all j, and the range check on W_{s} was performed at proposal time. Conversely, an accepted certificate yields, for each coin, a covering leaf whose record passes Local. □

Theorem 19 (Optimistic acceptance). *Assume Assumptions A1 to A4 and the timing conditions above. Then the following hold. (Safety) The probability that* Vault *finalizes a proposal whose* M_e *meets the winning condition of Definition 1, with the attestor list committed by the proposal's snapshot of* R_{att}*, is at most the bound of Theorem 5 plus* $\delta_{\mathsf{col}} = (12t^3 + 2q_C)/2^{h_C}$*, where* $q_C = 2k_{\mathsf{rev}}(\lceil \log_2 k_{\mathsf{rev}} \rceil + 2d + 7)$ *and* h_C *is the smallest output width of* C_{cert}*,* R_{sig} *and* R_{att} *(*$h_C = 256$ *in the deployed vault). Since* Col *ranges over the whole oracle,* δ_{col} *also excludes two key contracts with the same code hash* η*, and, in Theorem 20, two headers with the same* M_e *or two withdrawal trees with the same root. (Liveness) An honest relayer's proposal is never rejected except with probability* δ_{col}*; it is eligible for finalization at* $\tau_0 + T_{\mathsf{ch}}$ *if no dispute is raised, and by* $\tau_0 + T_{\mathsf{fin}}$ *otherwise; if the relayer then submits* finalize*, it is finalized within a further* Δ_{inc}*. (Cost) Each frivolous coin challenge costs its sender* b_C*, and an honest watcher spends at most one open-all transaction fee and one bond* b_C *per proposal; if its challenge is the first on a failing record, the bond is returned with* b_R*.*

Proof. Two different records that authenticate at the same index of C_{cert}, or two different leaves that open at the same position of R_{sig} or R_{att}, contain a collision; by Lemma 1 this has probability at most δ_{col}, and we exclude it. The *committed records* $(r_j)_{j<k_{\mathsf{rev}}}$ are then the unique records that authenticate under C_{cert}.

Safety. Suppose the vault finalizes a proposal whose M_e meets the winning condition. By Assumption A4, an honest watcher observes the proposal and, within Δ_{chk}, either evaluates Local on blob records that authenticate under C_{cert} or finds them missing or inconsistent. In the latter case it calls challengeAll, which is included by $\tau_0 + \Delta_{\mathsf{chk}} + \Delta_{\mathsf{inc}} < \tau_0 + T_{\mathsf{ch}}$ by Assumption A1, and hence accepted. Finalization requires that any open-all posting completed, since otherwise timeoutAll would have rejected the proposal, and then the watcher evaluates Local on the posted records within Δ_{chk} of τ_{post}. Either way, the watcher learns all committed records before the challenge deadline. If one of them, r_{j^*}, fails Local,

index j^* is challenged strictly before the deadline, by the watcher or by another party, since the watcher's challenge is included within $\Delta_{\mathsf{chk}} + \Delta_{\mathsf{inc}}$ of the proposal or of τ_{post}, which is less than T_{ch} or Δ_{post}. The only response that does not revert carries r_{j^*} and rejects the proposal, and an unanswered challenge rejects it at timeout; finalization waits for this outcome. Hence every committed record passes Local, and by Lemma 18 they form a certificate that Algorithm 1 accepts for M_e. An adversary that achieves this wins CERT-FORGE, which Theorem 5 bounds. The vault does not authenticate challengers, and Assumption A3 protects only the watcher's bond.

Liveness and cost. An honest relayer publishes records that authenticate and pass Local. Every response it gives to a coin challenge is accepted, and responses by other parties either revert or close the challenge. It posts all records within Δ_{open} if an open-all is requested. Its responses and postings are included by their inclusive deadlines because $\Delta_{\mathsf{resp}}, \Delta_{\mathsf{open}} \geq \Delta_{\mathsf{prep}} + \Delta_{\mathsf{inc}}$. No rejection condition can then occur, and the proposal is eligible for finalization, or becomes stale if another proposal for its epoch finalized first, once its deadlines have passed, which happens by $\tau_0 + T_{\mathsf{fin}}$. The cost statements follow from the bond rules and the once-per-proposal limit on open-all. □

Theorem 20 (Custody safety). *Assume Assumptions A1 to A4, the timing conditions above, and, in every epoch e, corrupt attestor stake below $W^e/3$ with $W^e_{\mathsf{p}} = \lfloor W^e/3 \rfloor$. Assume also the honest-signing rule of Section 5, and let* Fin *be the event that two honest validators consider conflicting checkpoints of the same epoch final. Then over E epochs, for every QPT adversary making t oracle queries in total,*

$$\Pr[\textit{G1 violated}] \leq E\Big(12t^2\varepsilon^* + \frac{48t^3 + 2q_V}{2^h} + \delta_{\mathsf{col}}\Big) + 2E\mathsf{Adv}^{\mathsf{uf}}_{N_{\mathsf{tot}}} + \mathsf{Adv}^{\mathsf{uf}}_{U} + \Pr[\mathsf{Fin}],$$

where $\varepsilon^* \leq 2^{-\lambda}/(12t^2)$ under Eq. (5) when $t \leq q(1 + 2^{-20})$ (Section 6.4), N_{tot} is the number of honest attestor keys generated over the whole experiment, and U is the number of user keys.

Proof. We show by induction on e that, outside the bad events below, the following invariant I_e holds after the vault finalizes its epoch-e checkpoint: (a) the finalized $M_1, \ldots, M_e$ are the checkpoints of the honest finalized sidechain; (b) the vault's (R_{att}, W, N) is the honest $(R^{e+1}_{\mathsf{att}}, W^{e+1}, N^{e+1})$; and (c) every payment made by withdraw so far pays a leaf of some finalized $\omega_{e'}$, each leaf at most once. I_0 holds by deployment: R^1_{att} is fixed in the constructor, and nothing has been paid.

Assume I_{e-1}, and consider the first proposal for epoch e that the vault finalizes. Every proposal for epoch e is opened while $e^* = e - 1$, so its snapshot of $(R_{\mathsf{att}}, W, W_{\mathsf{p}}, N)$ is the honest one of epoch e by I_{e-1}(b), and its M_e is computed by the vault over M_{e-1} and the proposed header. The attestors of epoch e are still bonded at that time by our unbonding condition, so their corrupt stake is below $W^e/3$ and hence at most W^e_{p}. If no honest attestor signed M_e, the winning condition of Definition 1 holds for this list. Consider the experiment truncated at this finalization, in which $\mathcal{A}$ makes at most t queries. By Theorem 19, on the direct path by Theorem 5, this happens with probability at most $12t^2\varepsilon^* + (48t^3 + 2q_V)/2^h + \delta_{\mathsf{col}} + 2\mathsf{Adv}^{\mathsf{uf}}_{N_{\mathsf{tot}}}$, where all honest attestor keys generated in the experiment form the multi-user forgery game ($K = N_{\mathsf{tot}}$). Otherwise some honest attestor signed M_e, so by the honest-signing rule M_e is final in its view, extends M_{e-1} and satisfies (i)–(iv). Outside Fin, M_e is the honest epoch-e checkpoint. Since the vault computed M_e from the header it stores, its ω_e, R^{e+1}_{att}, W^{e+1} and N^{e+1} are the honest ones, which gives I_e(a) and (b). For (c), withdraw pays only a leaf that opens under a finalized $\omega_{e'}$, and it marks the nullifier spent before transferring, so a leaf is paid at most once even across checkpoints and reentrant calls. A union bound over the E epochs gives the first term.

Under I_E, every paid leaf is a leaf of an honest checkpoint. By rules (ii) and (iii), it stems from a withdrawal executed on the honest sidechain, which debited its amount from its owner's balance and carries the owner's ML-DSA signature on the full operation, including recipient, amount, nonce and vault. Producing such a signature without the owner is a forgery against one of the U user keys. Deposits are credited once, and transfers and withdrawals conserve balances, so the total paid never exceeds the total credited deposits, which never exceed the escrow. The binding registry is not on the release path. Hence G1 can fail only through the stated events. □

S5.5 Remarks

The event Fin is the safety of the sidechain's finality gadget, which we do not re-analyze; execution validity is not part of it, because it is enforced by the honest-signing rule.

S6 Evaluation: Additional Details

This section complements Section 9 of the main paper.

S6.1 Artifacts

(i) `pqcert`, a Rust crate implementing Algorithm 1 with the QROM rule (Eq. (5)), domain-separated Keccak-256 Merkle trees with compact multiproofs, and 128-bit coin reduction. It uses the `crystals-dilithium` 1.0.0 crate used by the prototype of [38], which implements round-3 Dilithium rather than FIPS 204, and `pqcrypto-falcon`. Round-3 Dilithium2 has the same key and signature sizes as ML-DSA-44, and Dilithium3/5 signatures are 16 and 32 bytes shorter than ML-DSA-65/87; the algorithms are not interoperable. Results produced with this crate are labeled Dilithium in the main paper. A FIPS 204 backend (RustCrypto `ml-dsa` 0.1.1) cross-checks the headline results. (ii) A Python reference implementation (`pycert`) with the same logic over SHA3-256, using the FIPS 204 ML-DSA bindings of `pqcrypto`. (iii) A multi-process network harness. (iv) Monte Carlo simulators for QPoS-S and for grinding. (v) The Solidity vault of Section 9.5, tested with Foundry against FIPS 204 signatures. Code, raw results, plotting scripts and reproduction scripts are available at https://github.com/hassankhodaiemehr/PQ-Vault [21]; all experiments use fixed seeds.

S6.2 Machines

M1: a cloud VM with 2 vCPUs (Intel Xeon, 2.8 GHz) and 8 GB RAM. *M2*: a laptop VM with 2 vCPUs on an Intel Core i9-13900HX and 3 GB RAM, used as a heterogeneous light client. *M3*: 2 vCPUs (Intel Xeon, 2.1 GHz), used for the FIPS 204 cross-check. All timings are single-threaded medians.

S6.3 Harness Notes

Negative tests. Every benchmark run includes negative tests: a tampered signature, a shifted interval, a dropped reveal, and a claimed signed weight above W. The verifier rejected all of them in every run. The checks $|P| \leq k_{\mathsf{rev}}$ and "no unused reveal" were added after the size runs; unit tests and the FIPS 204 cross-check exercise them.

Proven threshold in the size runs. The size benchmarks used $W_{\mathsf{p}} = \lfloor 0.3333\, W \rfloor$, which yields the same k_{rev} as $\lfloor W/3 \rfloor$ in all 40 reported configurations; the contract and network experiments use $\lfloor W/3 \rfloor$. The harness now uses $\lfloor W/3 \rfloor$ too; this changes the coins, so a rerun gives slightly different reveal sets (e.g., 81 instead of 94 distinct reveals at $N = 2^8$), and all reported sizes are from the 0.3333 runs.

S6.4 Network Experiment Setup

An aggregator process and two worker processes, each hosting about $N/2$ validators, communicate over real TCP sockets, with one connection per validator and real FIPS 204 ML-DSA-44 signatures. The aggregator verifies every signature, builds the certificate, and sends it to 8 light clients, which verify it. WAN conditions are emulated per message. Validators are placed in five regions with one-way latency to the aggregator of 1, 35, 40, 60 and 95 ms (with weights 0.30, 0.15, 0.30, 0.05, 0.20), with 10% log-normal jitter, 50 Mbps validator uplinks, and a 1 Gbps aggregator link. 10% of the validators are offline. Because all validators share two vCPUs, signing is charged as the measured single-core signing time of round-3 Dilithium2 in `pqcert` (0.42 ms), while the signatures themselves are real FIPS 204 signatures; all aggregator and light-client work runs for real. The aggregator stops collecting at 80% of the stake.

S6.5 Contract Implementation and Tests

Gas was measured with Foundry [50] (forge 1.5.1, solc 0.8.30, Osaka EVM rules). Each attestor registers its expanded ML-DSA-44 key once as an SSTORE2 contract, and its attestor leaf commits to the `EXTCODEHASH` of that contract, which the vault checks during a dispute. The test vectors are a real certificate over an attestor list of $N = 2^{16}$ positions ($d = 16$), bound in M_e as N^{e+1}. Of these, 1024 carry positive weight and the rest have weight zero, which Local never accepts because it requires $w_{i_j} > 0$; the direct and local verifiers use the same N and d. The vectors include 872 FIPS 204 signatures produced with the ZKNOX reference signer (in deterministic mode, for reproducible vectors), $k_{\mathsf{rev}} = 193$ coins and 130 distinct reveals. Both verifiers accept these FIPS 204 signatures. Before signing R_{att}^{e+1}, honest validators check that each registered expanded key is the expansion of the committed pk_i, and the on-chain check therefore equals $\mathsf{ML\text{-}DSA.Vf}(pk_i, \cdot)$. The 17 tests per verifier also confirm the state machine: a record that fails Local rejects the proposal; a tampered record, a wrong path and a wrong key address supplied by a third party revert without rejecting; finalization waits for open challenges, and a late unanswered challenge rejects; an open-all request is either satisfied or rejects; a rejected epoch can be proposed again; a substituted ω_e is rejected; the record count must equal k_{rev} exactly; and a withdrawal is paid only once. The packed verifier of the ZKNOX experimental branch uses an external Keccak-f[1600] helper contract.

S6.6 Gas Accounting and Extrapolation

Table 8 reports transaction-level gas under EIP-7623 [45], $21{,}000 + \max(G_{\mathsf{exec}} + 4T,\ 10T)$, where G_{exec} is the execution gas measured by the test and $T = n_0 + 4n_1$ counts the zero and nonzero calldata bytes. Gas is measured inside Foundry tests, where storage written earlier in a test is warm. All operations fit in one transaction except direct verification, which exceeds the 2^{24}-gas cap [46], and the open-all posting, which is split to respect the 128 KB mempool limit. The key-registration total assumes nonzero calldata bytes. The two verifiers are the NIST-conformant verifier of the ZKNOX main branch and the packed verifier of its experimental branch. The propose and finalize rows of Table 8 sum to 4.2×10^5, and the single happy-path test measures 4.1×10^5 with either verifier. For direct verification, dividing the exact costs (1.08×10^9 and 1.79×10^8) by the 2^{24} cap gives 64.2 and 10.6, hence at least 65 and 11 transactions. At $N = 2^{16}$ with 189 distinct reveals (Table 5), the measured cost per distinct reveal (8.3×10^6 NIST, 1.4×10^6 packed) extrapolates to 1.6×10^9 and 2.6×10^8 gas. An open-all posting uses a different serialization from the compact certificate: each of the 193 records carries its signature and two single-leaf paths (4,160 bytes in ABI encoding), for 803,580 bytes in 7 transactions under the 128 KB limit. Its 3.0×10^7 gas is set almost entirely by the calldata floor; execution adds only 1.6–2.0×10^6.

S7 Extended Discussion

This section gives the full version of the discussion in Section 10 of the main paper.

S7.1 Scope of the L1 Assumption

PQ-Vault protects assets against the failure of *account-level* cryptography. A CRQC that breaks ECDSA also breaks the BLS signatures of Ethereum's consensus, and Assumption A1 is therefore not implied by our threat model: it assumes that L1 consensus is not attacked, or is restored by a hard fork that preserves contract state [4]. Such an attack is publicly visible and would meet a social response, and after such a fork vault balances remain spendable only through PQ predicates, whereas EOAs need special recovery. For the same reason, validators credit deposits from full L1 nodes, not from BLS-based light clients. Assumption A1 also bounds censorship: T_{ch} must exceed the time for which block producers can be bribed to exclude a challenge, which motivates our seven-day window. Assumption A3 is needed only for transactions that expose an ECDSA key: deposits, initial bindings, and the bonds of relayers and watchers. Private submission channels reduce the exposure. PQ smart accounts for relayers and watchers would remove it.

S7.2 Data Availability and Watcher Incentives

Theorem 19 assumes that one honest watcher is online. Withholding blob data cannot cause a false acceptance (Section 8). Bonds follow from Table 8 for a gas-price ceiling $p^{\max}$: $b_C \geq 8.2 \times 10^6 p^{\max}$ covers a response, and $b_R \geq b_C$ plus a reward. Open-all requests carry no bond, so withholding cannot drain honest watchers: per proposal, a watcher spends at most one request fee and one bond b_C (Assumption A4), which it recovers with a reward when it exposes a failing record. The price is that anyone can force each proposal to post its records once, which costs the relayer 3.0×10^7 gas against about 2.4×10^4 for the request; relayers must recover this cost through fees, and G4 relies on their doing so. The prototype implements the vault and dispute logic but not the binding registry or the canary, and its reveal rule targets $\lambda = 128$ per checkpoint rather than a lifetime budget (Section 6.4). Bonds deter griefing, not theft; safety rests on Assumption A4, and any depositor can watch at about 20 ms per checkpoint (18 ms with round-3 Dilithium2, 21 ms with FIPS 204). A full incentive analysis, including relayer compensation, is left open. Blob (KZG) commitments must never be on the acceptance path (Proposition 17).

S7.3 Public Leader Schedule

QPoS-S fixes and publishes each epoch's leaders when the seed is final. This makes the quantum proof of Theorem 15 possible: freshness rests on one hidden honest secret, and binding is statistical in the QROM. It also reveals each epoch's leaders as soon as the epoch begins, and an adversary with late reveals sees all its candidate schedules. QPoS-S limits both effects. With $L = 3$ fallback proposers per slot, an adversary that suppresses B honest proposers per slot after seeing the schedule removes the honest proposer only from slots that are not guaranteed: with probability 0.3 per slot at $\alpha = 0.3$ without denial of service and 0.514 with $B = 1$, so $n = 31$ and $n = 56$ slots suffice for inclusion except with probability $2^{-40} + 2E(\epsilon_{\mathsf{LE}} + \delta_{\mathsf{LE}})$ (Theorem 11 and Corollary 16). Without fallback proposers, a single suppressed leader per slot would halt the sidechain. The stake-weighted random reveal order, keyed by the previous seed, makes late adversarial reveals rare ($\Pr[c \geq x] \leq \mu\alpha^x$, Proposition 13). Without denial of service, the target-slot probability at $\alpha = 0.3$ is then at most 0.622, against 0.997 for an adversary that owns the last four reveals, but still above the 0.348 of the single-slot runner-up attack on hash-chain PoS; for $\alpha \leq 0.2$ it is at most 0.265. Inclusion of honest reveals is proved (Lemma 12), given that the finality gadget finalizes each block of the reveal window within its slot. Instant adaptive corruption, which could seize the last honest revealer once the order is known, is outside our model; corruptions take effect three epochs after they are requested. Hidden schedules need a PQ verifiable random function or secret leader election, whose quantum analysis we leave open. Because PQ-Vault attaches no exclusive role to a slot, a censored or attacked leader delays the sidechain but not certification.

S7.4 Limitations of the Evaluation

The sidechain is analyzed by simulation only. The network experiment emulates WAN latency on shared cores rather than running on geo-distributed hosts, with five rounds per configuration, and we did not inject relayer or data-withholding faults end to end. If the corrupt third abstains and 85% of honest stake signs, $W_{\mathsf{s}}/W_{\mathsf{p}} = 1.7$ and $k_{\mathsf{rev}} = 340$, which enlarges certificates but leaves the optimistic L1 path unchanged. Gas was measured in a local Foundry EVM, not on a public network, and the hash-based baseline was composed from measured components rather than run end to end. Our sidechain relies on a standard BFT finality gadget that we did not re-analyze, and the leader-election bounds assume that the gadget finalizes each block of the reveal window within its slot; inclusion of honest reveals is then proved (Lemma 12). All our bounds hold in the QROM with ML-DSA's internal hash modeled as an independent oracle. A fully succinct PQ acceptance path would prove certificate verification inside a transparent, hash-based proof system; the prototype circuit of [38] for the Dilithium key relation ($8{,}192$ multiplication gates, 140 KB proof, 0.7 s proving) indicates that this is feasible but costly today. These are natural targets for future work, together with a formally verified vault contract. Our analysis applies to any EUF-CMA signature scheme. Falcon-512 halves certificate size (Section 9.2) but needs floating-point Gaussian sampling, which is hard to implement in constant time, and SLH-DSA signatures are much larger. Combining certificates with synchronized hash-based multi-signatures [6] is a promising direction.

ACKNOWLEDGMENTS

We thank Saeid Yazdinejad for his simulation work on the design of [18].

REFERENCES

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
[2] D. Aggarwal, G. K. Brennen, T. Lee, M. Santha, and M. Tomamichel, "Quantum attacks on Bitcoin, and how to protect against them," *Ledger*, vol. 3, Oct. 2018.
[3] I. Stewart, D. Ilie, A. Zamyatin, S. Werner, M. F. Torshizi, and W. J. Knottenbelt, "Committing to quantum resistance: A slow defence for Bitcoin against a fast quantum computing attack," *Royal Society Open Science*, vol. 5, no. 6, p. 180410, 2018.
[4] V. Buterin, "How to hard-fork to save most users' funds in a quantum emergency," Ethereum Research forum, https://ethresear.ch/t/18901, 2024.
[5] R. Dubois and S. Masson, "EIP-8051: Precompiles for ML-DSA signature verification," Ethereum Improvement Proposals, draft, https://eips.ethereum.org/EIPS/eip-8051, 2025.
[6] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, "Hash-based multi-signatures for post-quantum Ethereum," *IACR Communications in Cryptology*, vol. 2, no. 1, Apr. 2025.
[7] Quantum Resistant Ledger Foundation, "QRL proof-of-stake algorithm," https://www.theqrl.org/features/proof-of-stake/, 2017.
[8] Abelian Foundation, "Abelian: A quantum-resistant cryptocurrency," https://www.abelianfoundation.org, 2019.
[9] A. Holcomb, G. Pereira, B. Das, and M. Mosca, "PQFabric: A permissioned blockchain secure from both classical and quantum attacks," in *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*, 2021, pp. 1–9.
[10] S. Dolev, B. Guo, J. Niu, and Z. Wang, "SodsBC: A post-quantum by design asynchronous blockchain framework," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 1, pp. 47–62, 2024.
[11] ZKNOX, "ETHDILITHIUM: ML-DSA verification in Solidity," https://github.com/ZKNoxHQ/ETHDILITHIUM, 2025.
[12] ——, "ETHFALCON: Falcon verification in Solidity," https://github.com/ZKNoxHQ/ETHFALCON, 2025.

[13] P. Gaži, A. Kiayias, and D. Zindros, "Proof-of-stake sidechains," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2019, pp. 139–156.

[14] S. Micali, L. Reyzin, G. Vlachos, R. S. Wahby, and N. Zeldovich, "Compact certificates of collective knowledge," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2021, pp. 626–641.

[15] Algorand Foundation, "State proofs," Algorand developer documentation, https://developer.algorand.org/docs/get-details/stateproofs/, 2022.

[16] P. Chaidos and A. Kiayias, "Mithril: Stake-based threshold multisignatures," in *Proc. CANS 2024*, ser. LNCS, vol. 14905, 2025, pp. 239–263.

[17] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, "zkBridge: Trustless cross-chain bridges made practical," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 3003–3017.

[18] H. Khodaiemehr, K. Bagheri, S. Yazdinejad, and C. Feng, "NIROPoK-based post-quantum sidechain design on Ethereum," Cryptology ePrint Archive, Paper 2026/401, 2026.

[19] J. Zhang, T. Xie, Y. Zhang, and D. Song, "Transparent polynomial delegation and its applications to zero knowledge proof," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 859–876.

[20] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.

[21] H. Khodaiemehr, K. Bagheri, and C. Feng, "PQ-Vault: Code, raw results and reproduction scripts," https://github.com/hassankhodaiemehr/PQ-Vault, 2026.

[22] Abelian Foundation, "Post-quantum zero-knowledge (PQZK) bridge," https://docs.abelian.info/docs/technical-papers/pqzk-bridge/, 2023.

[23] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, ser. LNCS, vol. 9666, 2016, pp. 305–326.

[24] E. Ben-Sasson, A. Chiesa, and N. Spooner, "Interactive oracle proofs," in *Theory of Cryptography Conference (TCC)*, ser. LNCS, vol. 9986, 2016, pp. 31–60.

[25] N. Fleischhacker, G. Herold, M. Simkin, and Z. Zhang, "Chipmunk: Better synchronized multi-signatures from lattices," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2023, pp. 386–400.

[26] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 1353–1370.

[27] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling Byzantine agreements for cryptocurrencies," in *Proc. 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 51–68.

[28] B. David, P. Gaži, A. Kiayias, and A. Russell, "Ouroboros Praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain," in *Advances in Cryptology – EUROCRYPT 2018*, ser. LNCS, vol. 10821, 2018, pp. 66–98.

[29] M. F. Esgin, V. Kuchta, A. Sakzad, R. Steinfeld, Z. Zhang, S. Sun, and S. Chu, "Practical post-quantum few-time verifiable random function with applications to Algorand," in *Financial Cryptography and Data Security (FC)*, ser. LNCS, vol. 12675, 2021, pp. 560–578.

[30] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Advances in Cryptology – CRYPTO 2017*, ser. LNCS, vol. 10401, 2017, pp. 357–388.

[31] P. Gaži, A. Kiayias, and A. Russell, "Stake-bleeding attacks on proof-of-stake blockchains," in *Crypto Valley Conference on Blockchain Technology (CVCBT)*, 2018, pp. 85–92.

[32] D. Boneh, Ö. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry, "Random oracles in a quantum world," in *Advances in Cryptology – ASIACRYPT 2011*, ser. LNCS, vol. 7073, 2011, pp. 41–69.

[33] J. Don, S. Fehr, C. Majenz, and C. Schaffner, "Security of the Fiat–Shamir transformation in the quantum random-oracle model," in *Advances in Cryptology – CRYPTO 2019*, ser. LNCS, vol. 11693, 2019, pp. 356–383.

[34] A. Chiesa, P. Manohar, and N. Spooner, "Succinct arguments in the quantum random oracle model," in *Theory of Cryptography Conference (TCC)*, ser. LNCS, vol. 11892, 2019, pp. 1–29.

[35] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," NIST, Federal Information Processing Standards Publication FIPS 204, 2024.

[36] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.

[37] M. Zhandry, "How to record quantum queries, and applications to quantum indifferentiability," in *Advances in Cryptology – CRYPTO 2019*, ser. LNCS, vol. 11693, 2019, pp. 239–268.

[38] H. Khodaiemehr, K. Bagheri, S. Yazdinejad, and C. Feng, "Post-quantum sidechain prototype for Ethereum," https://github.com/Xm0onh/NIROPoK-PostQuantum-Sidechain, 2025.

[39] V. Buterin, S. Wilson, A. Dietrichs, and lightclient, "EIP-7702: Set code for EOAs," Ethereum Improvement Proposals, no. 7702, https://eips.ethereum.org/EIPS/eip-7702, May 2024.

[40] National Institute of Standards and Technology, "Submission requirements and evaluation criteria for the post-quantum cryptography standardization process," https://csrc.nist.gov/csrc/media/projects/post-quantum-cryptography/documents/call-for-proposals-final-dec-2016.pdf, 2016.

[41] G. Brassard, P. Høyer, and A. Tapp, "Quantum cryptanalysis of hash and claw-free functions," in *LATIN'98: Theoretical Informatics*, ser. LNCS, vol. 1380. Springer, 1998, pp. 163–169.

[42] P. S. Efraimidis and P. G. Spirakis, "Weighted random sampling with a reservoir," *Information Processing Letters*, vol. 97, no. 5, pp. 181–185, 2006.

[43] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, "Strengths and weaknesses of quantum computing," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1510–1523, 1997.

[44] B. Wagner *et al.*, "hash-sig: Rust implementation of synchronized hash-based signatures," https://github.com/b-wagn/hash-sig, 2025.

[45] T. Wahrstätter and V. Buterin, "EIP-7623: Increase calldata cost," Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-7623, 2024.

[46] G. Rebuffo and T. Wahrstätter, "EIP-7825: Transaction gas limit cap," Ethereum Improvement Proposals, no. 7825, https://eips.ethereum.org/EIPS/eip-7825, Nov. 2024.

[47] Algorand, "go-algorand, file config/consensus.go," https://github.com/algorand/go-algorand, 2026.

[48] D. J. Bernstein, "Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete?" in *SHARCS'09 Workshop Record*, 2009, pp. 105–116.

[49] E. Deirmentzoglou, G. Papakyriakopoulos, and C. Patsakis, "A survey on long-range attacks for proof of stake protocols," *IEEE Access*, vol. 7, pp. 28 712–28 725, 2019.

[50] Foundry contributors, "Foundry: Ethereum development toolkit," https://github.com/foundry-rs/foundry, 2025.